

MAZE SURVIVOR

COMPUTER SCIENCE NEA

CRAIG HERSEY

MAZE SURVIVOR– Computer Science NEA

Contents:

Analysis.....	
<u>Application concept</u> – Page 2	
<u>Inspiration for application</u> - Page 3	
Research:	
<u>Programming language</u> - Page 4	
<u>Libraries</u> - Page 5	
<u>Maze generator algorithms</u> - Page 8	
<u>End User Survey</u> – Page 11	
<u>Practical Limitations</u> – Page 11	
<u>Current System Specifications</u> - Page 11	
<u>Objectives</u> - Page 12	
Design.....	
<u>Project Structure</u> - Page 14	
<u>Flowchart Diagram</u> - Page 15	
<u>Data Flow Diagram</u> - Page 16	
<u>Class Diagram</u> - Page 17	
<u>Class Structure</u> - Page 18	
<u>Data Structures</u> - Page 32	
<u>Algorithms</u> – Page 33	
<u>Graphical user interface</u> – Page 34	
Technical solution.....	
<u>File Structure</u> - Page 35	
Code:	
<u>MazeGeneration.py</u> - Page 36	
<u>DrawMaze.py</u> - Page 37	
<u>Main.py</u> = Page 45	
Testing.....	
<u>End Product Testing</u> - Page 63	
Evaluation.....	
<u>Application Evaluation</u> - Page 66	
<u>End User Feedback</u> - Page 68	
<u>Improvements for the Future</u> - Page 69	
.....	
<u>Bibliography</u> - Page 70	

MAZE SURVIVOR– Computer Science NEA

Analysis

Application concept:

The idea for my application is to make a game set in a randomly generated 2-D maze that involves two players, with one player being the runner and the other the being a tagger. The way the runner wins the game is surviving the tagger by navigating the maze. There is no escape to the maze, instead the runner must avoid getting tagged in a set time frame which is determined by the difficulty.

In order to generate the 2-D maze, I will need to find a maze generation algorithm that is grid based. I will also need to find a programming language that supports a GUI toolkit that will allow me to make simple visual of the maze, make simple sprites for the runner and the tagger and finally make menu for the game that allows the user to set the difficulty and start the game. I want the application to appeal to people who playing video games.

I'm making this application to see if I can expand my programming skills, one way I am doing this is programming Maze Survivor using Object-Oriented-Programming as it is a programming paradigm I am unfamiliar with and I would like to learn it.

MAZE SURVIVOR– Computer Science NEA

Inspiration for application:

The Amazing Maze Game

The amazing Maze Game is a maze arcade game that is considered to be one of the first ever maze games. The game involves the player having to escape a simple maze that was randomly generated using Prim's algorithm.

I will use The Amazing Maze Game as inspiration for my application for how I am my mazes to be generated

(Source: https://en.wikipedia.org/wiki/The_Amazing_Maze_Game)

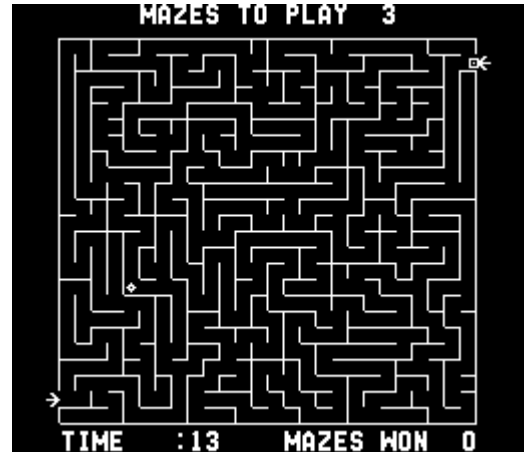


Figure 1: A game of The Amazing Maze Game in progress

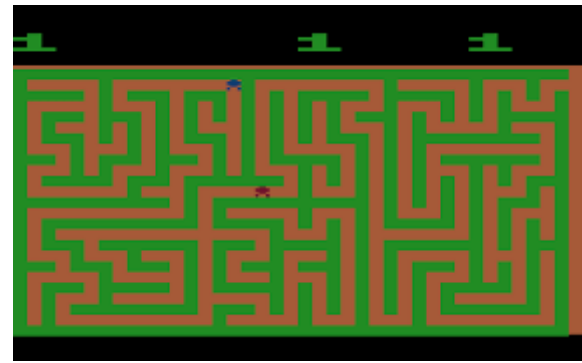
(Source: <https://en.wikipedia.org/w/index.php?curid=6005946>)

Maze Craze: A game of Cops n' Robbers

Maze Craze: A game of Cops n' Robbers is a two-player maze video game in which there is a robber that is being chased by a policeman in maze. The goal of the robber is to escape the maze before it gets caught by the policemen making this very similar to Maze Survivor

The maze is not generated using a maze generator algorithm and is the same each time the game is played. The maze also has an exit. Although my application's gameplay loop will be inspired by Maze Craze: A Game of Cops n' Robbers, I plan to have my application to be different to Maze Craze: A Game of Cops n' Robbers by having the maze not have an exit and have the maze randomly generate each game.

(Source: https://en.wikipedia.org/wiki/Maze_Craze)



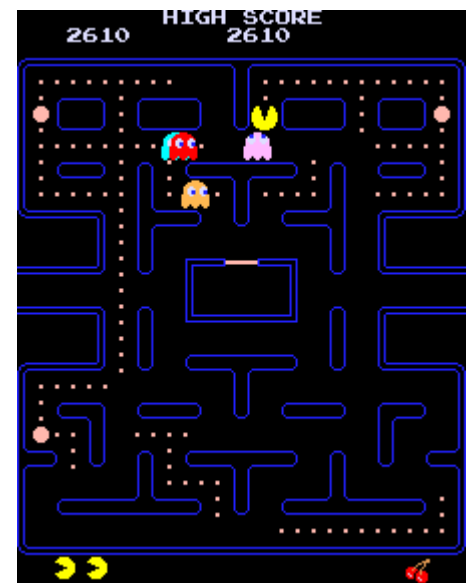
(Source: <https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcTHMs5542Xol-gkZqk>)
Figure 2: A game of Maze Craze: A game of Cops n' Robbers in progress

Pac-Man

Pac-Man is an arcade maze video game. You play as Pac-Man who is trying to navigate a maze trying to eat all the "power pellets" while trying to avoid the ghost are trying to catch Pac-Man making this also very similar to Maze Survivor.

The maze is also not generated using a maze generator algorithm and is the same each time the game is played. There is also the fact that Pac-Man is a one player game while Maze survivor requires two players. Pac-Man mazes also have "warp tunnels" which allow Pac-Man and the ghosts to travel to the opposite side of the maze. I plan have my application to be different from Pac-Man by making the application two player as well as making the maze have no quick travel points/tunnels

(Source: <https://en.wikipedia.org/wiki/Pac-Man>)



(Source: https://upload.wikimedia.org/wikipedia/commons/c/co/Pac-Man_ga)
Figure 3: A game of Pac-Man in progress

MAZE SURVIVOR– Computer Science NEA

Research:

Programming language:

In my research I had to find a programming language that I would use to program my application. The programming language I required would have to allow me to generate a grid-based 2-D maze, allow for me to make a simple GUI for a game menu and finally allow me to make a moving sprite for the runner and the tagger

My options for a programming language were between the programming languages that are allowed by AQA exam board:

- C#
- Java
- Python

C#:

C# is a general-purpose high-level programming language that uses object-oriented programming to write its programs. Advantages of using C# include it's being easy write to debug due to its clear syntax. It's also utilising automatic garbage collection which reducing memory leaks which may improve performance of the application. However, disadvantages of using C# include it doesn't have a built in GUI toolkit. I also have no experience with C#

(Source: [https://en.wikipedia.org/wiki/C_Sharp_\(programming_language\)](https://en.wikipedia.org/wiki/C_Sharp_(programming_language)))

(Source: <https://www.geeksforgeeks.org/c-sharp/introduction-to-c-sharp/>)

Java:

Java is a general-purpose, object-oriented programming language. Advantages of using Java include that has a built in GUI toolkit called Swing, it is also a straightforward language to program in making it easy to write and debug. However, disadvantaged of using Java include Java having slow execution which may affect the performance of the application. I also have no experience with Java

(Source: [https://en.wikipedia.org/wiki/Java_\(programming_language\)](https://en.wikipedia.org/wiki/Java_(programming_language)))

(Source: <https://www.geeksforgeeks.org/java/advantages-and-disadvantages-of-java/>)

Python:

Python is high-level, general-purpose programming language that can utilise both procedural and object-oriented programming to write its programs. Advantages of using Python include its versatility due to its support of both procedural and object-oriented programming. It also has a built-in library called Tkinter which is Python's built in GUI toolkit. I also have experience using Python. However, a disadvantage to using python includes the fact it is an interpreted language which means it can be slower than compiled languages like Java and C# which may affect the performance of the application

(Source: [https://en.wikipedia.org/wiki/Python_\(programming_language\)](https://en.wikipedia.org/wiki/Python_(programming_language)))

(Source: <https://www.geeksforgeeks.org/python/python-language-advantages-applications/>)

MAZE SURVIVOR– Computer Science NEA

I decided to use Python as my programming language for my application, one of the biggest reasons is because of its built-in Tkinter library which allows me to make the simple grid visual required to make the layout of the maze and create a simple main menu for the application. I also have the most experience with python and wish to use it for this project to help me expand my programming skills as I find it easy to follow.

Libraries:

Tkinter:

As I mentioned on of the biggest reasons for me using Python is because of Tkinter. Tkinter is a simple GUI tool kit built into Python which will allow me to make my 2-D maze that the players can move around as well as make the simple sprites that would represent the runner and the tagger which would both move around the maze. Tkinter will also allow me to make a simple main menu that can allow the player to start the game and set the game difficulty.

However, using Tkinter means I am limited in the use of iterative loops and programming practices in Maze Survivor, this is because the tkinter command `.mainloop()` runs a while loop to check for events within the window. This means that the main thread is occupied and no other while loops can be run. Therefore, when using Tkinter I will have to use recursive programming techniques

For Maze Survivor I will use the following Tkinter commands:

Command:	Function:
<code>tkinter.Tk()</code>	Initiates the main window, this is done by setting a variable equal to this command (e.g <code>window = tkinter.Tk()</code>) This then allows you, to change the size and title of window. Aswell as add buttons, labels and shapes inside the confinement of the main window.
<code>window.title('Title')</code>	Allows you to set the title of the window created using Tkinter by entering a string as a parameter, this will allow me to name my application Maze Survivor when the game is running.
<code>window.geometry('(width)x(height)')</code>	Allow you to change the size of the main window by entering the width and height as a parameter. The parameter syntax must be a string with an 'x' in between the width and the height (e.g '1920x1080').
<code>window.destroy()</code>	Allows you to destroy any objects made using Tkinter. This can be windows, canvas, or buttons.
<code>window.mainloop()</code>	Runs an infinite loop used to run the window, wait for an event to occur, e.g a key press to move the runner or tagger
<code>Label(master, options)</code>	Allows you to make a widget in the main window which can display text or images. The <i>master</i> parameter represents the main window, while the <i>options</i> can be a multitude of things, such height, width, font size and background colour. These options can be used

MAZE SURVIVOR– Computer Science NEA

	as key-value pairs separated by commas. This allows me to make menu backgrounds in Maze Survivor by using “image =” parameter as part of the label’s options. I will be able to put a variable which holds an image by using the PhotoImage command.
Button(<i>master,options</i>)	Allows you to make a button image which causes a command to be run when pressed. The <i>master</i> parameter represents the main window, while the <i>options</i> can be a multitude of things, such height, width, text on the button and the command that is executed when the button is pressed. These options can be used as key-value pairs separated by commas. This allows me to make a functional menu in Maze Survivor by using the “command =” and “image =” parameters as part of the button’s options. I will be able to enter a class method in the “command” parameter as well as able to put a variable in the image parameter which holds an image by using the PhotoImage command.
widget.place(<i>x, y</i>)	Allows you explicitly to set the position of a widget. The parameters <i>x</i> and <i>y</i> are the coordinates within the window or where you want the widget to be placed.
PhotoImage(file = “ <i>file path</i> ”)	Allows you to store an image in a variable that takes the file path of the image as a parameter. The variable which stores the photo can then be used with the Label() and Button() command with its image parameter to display the image in the main window.
C = Canvas(<i>master, height, width, options</i>)	Creates an area in the main window which allows you to draw shapes. The <i>master</i> parameter represents the main while the <i>height</i> and <i>width</i> parameters represent how big the canvas is. The <i>options</i> parameter can be a multitude of things such as background colour and border colour. These options can be used as key-value pairs separated by commas. This will allow me to draw squares to represent the maze as well as make the sprites that represent the “Runner” and the “Tagger”.
C.create_rectangle(<i>xo,yo, x1, y1, options</i>)	A Canvas method that allows you to create a rectangle within the canvas area. The <i>x</i> and <i>y</i> parameters represent coordinates on the canvas. This allows the size of the rectangle to be determined. By having the distance and height be the same you can use this function to make a square. The <i>options</i> parameter can be a multitude of things such as the colour of the rectangle. Using this command, I can represent a path and a wall in the maze as a different colour squares as well as making squares which represent the “Runner” and “Tagger” sprites.
C.move(<i>canvasObject,x,y</i>)	A Canvas method that allows you to move objects within the canvas area. The parameter <i>canvasObject</i> represents the object you would like to move. The <i>x</i> and <i>y</i> parameters represent the coordinates of which how much you move the

MAZE SURVIVOR– Computer Science NEA

	object. This allows me to make it, so the “Runner” and “Tagger” sprites have the functionality to move around the maze.
<code>widget.bind(“key”, command)</code>	Allows you to setup keybinds. This means that you can run a function when a key is pressed. This will allow me to make the “Runner” and “Tagger” move around the maze.
<code>widget.unbind(“key”)</code>	Allows you to unbind the keybind from the widget it is bound to, so the key no longer runs a function. This will allow me to make sure the “Runner” and “Tagger” don’t move when the game is finished
<code>widget.focus_set()</code>	A method that activates the keybinds in the widget it is bound to
<code>widget.after(master, ms, function, arguments)</code>	Allow you to to schedule a function to be executed after a set delay in milliseconds. The parameter <i>master</i> is the main window or widget using the .after command. The <i>ms</i> parameter is the delay in milliseconds. The <i>function</i> parameter is the to be executed after the delay and the <i>arguments</i> parameter is any of the parameters that go into <i>function</i> . This will allow me to make a timer using recursive method with said method calling on itself each time a second goes by, deducing the amount of time left shown on the timer

Turtle:

Turtle is a library on Python that allows you control a “turtle” that acts as a pen as sorts which allows you to draw shapes This means I could use Turtle to make the GUI for my application. However, Turtle doesn’t have the capability to use text or import images like Tkinter. Therefore, Turtle will not be used to make the GUI for Maze Survivor

Random:

Random is also a library on Python that allows for the use of random number generation Using this library will be essential as it will allow me to randomly generate the mazes in the game

SYS:

SYS is a Python library that allows you to perform file directories. This will allow me to do mulit-file handling in my project which will allow me to challenge my programming skills to familiarise myself with programming across different files and making them work together

MAZE SURVIVOR– Computer Science NEA

Maze Generator Algorithms:

In my research I had to find a maze generation algorithm that allowed me to make a 2-D maze that would be able to be generated using Tkinter.

There were many options of maze generation algorithm to choose from. I was not that particular on the efficiency of the algorithms and was more focus on their practicality and complexity and how each of them generates a maze. I wanted the algorithm to generate a maze that would not have many long winding paths as it would remove some excitement from playing the game.

I narrowed my choices between the following algorithms:

- Kruskal's algorithm
- Prim's algorithm
- Eller's algorithm

Kruskal's algorithm:

Using Kruskal's algorithm to generate a maze, it first sets every single cell in the grid to be an individual set. It then selects an edge between two cells at random and combines the two cells to be under the same set and become connect cells. It continues to select the edges between two cells and combines them until there no longer any cells to consider.

Overall, this makes Kruskal's one of the harder algorithms to develop, and I find that the way it picks it edges will make it too complex to work for a grid-based algorithm

(Source: <https://weblog.jamisbuck.org/2011/1/3/maze-generation-kruskal-s-algorithm>)

Kruskal's algorithm Pseudocode:

```
function Kruskal(Graph G) is
    F := ∅
    for each v in G.Vertices do
        MAKE-SET(v)

    for each {u, v} in G.Edges ordered by increasing weight({u, v}) do
        if FIND-SET(u) ≠ FIND-SET(v) then
            F := F ∪ { {u, v} }
            UNION(FIND-SET(u), FIND-SET(v))
    return F
```

(Source: https://en.wikipedia.org/wiki/Kruskal%27s_algorithm)

MAZE SURVIVOR– Computer Science NEA

Prim's algorithm:

Using Prim's algorithm to generate a maze, it picks a cell in the grid and sets all of those adjacent cells and sets them as frontier cells, one of the frontier cells are select at random and make a path for the maze. Then all the adject cells to the previous randomly selected frontier cell now become new frontier cells. This process is repeated until the maze has been generated

Overall this makes Prim's one of the easier algorithms to develop. It's way of generating Maze's is perfect for making a grid-based algorithm

(Source: <https://weblog.jamisbuck.org/2011/1/10/maze-generation-prim-s-algorithm>)

Prim's algorithm Pseudocode:

```
function Prim(vertices, edges) is
  for each vertex in vertices do
    cheapestCost[vertex]  $\leftarrow \infty$ 
    cheapestEdge[vertex]  $\leftarrow$  null

  explored  $\leftarrow$  empty set
  unexplored  $\leftarrow$  set containing all vertices

  startVertex  $\leftarrow$  any element of vertices
  cheapestCost[startVertex]  $\leftarrow$  0

  while unexplored is not empty do
    // Select vertex in unexplored with minimum cost
    currentVertex  $\leftarrow$  vertex in unexplored with minimum
    cheapestCost[vertex]
    unexplored.remove(currentVertex)
    explored.add(currentVertex)

    for each edge (currentVertex, neighbor) in edges do
      if neighbor in unexplored and weight(currentVertex,
neighbor) < cheapestCost[neighbor] then
        cheapestCost[neighbor]  $\leftarrow$  weight(currentVertex, neighbor)
        cheapestEdge[neighbor]  $\leftarrow$  (currentVertex, neighbor)

  resultEdges  $\leftarrow$  empty list
  for each vertex in vertices do
    if cheapestEdge[vertex]  $\neq$  null then
      resultEdges.append(cheapestEdge[vertex])

  return resultEdges
```

MAZE SURVIVOR– Computer Science NEA

Eller's algorithm:

Using Eller's algorithm to generate a maze, first it selects the cells in the first row of the maze and makes each cell an individual set. It then joins adjacent random cells that aren't in the same set and merges them into a single set. Each set then randomly creates vertical connections downwards to the next row of the grid. The cell selected from the vertical connection then joins that set.

Any remaining cells in the next row that weren't vertically connected become their own sets. This process is then repeated until it reaches the last row in the grid in which the last row sets all the cells to be in the same set.

Overall, this makes Eller's algorithm the hardest and most complex algorithm to develop. I find that it's way too complex to put into a grid-based algorithm.

(Source: <https://weblog.jamisbuck.org/2010/12/29/maze-generation-eller-s-algorithm>)

Eller's algorithm Pseudocode:

Step 1: Initialize the first row.

Step 2: For each column in that row, if it doesn't have a cell, create one and add it to its own set. Each cell in the row should be part of a set at this point.

Step 3: Going from left to right, if the current cell and the next cell are in different sets, randomly decide to connect to the next cell. If they connect, merge the cells' sets together.

Step 4: Initialize the next row. For each set in the current row, connect downward for at least one cell per set, adding that cell to the set it connected from.

Step 5: Move to the next row and repeat steps 2 through 4.

Step 6: When on the last row, repeat step 3 but remove the randomness, always connecting to the next cell if it's in a different set. Don't make any downward connections.

(Source: <https://cantwell-tom.medium.com/ellers-maze-algorithm-in-javascript-2e5742c1a4cd>)

In the end I decided to choose Prism's algorithm due to it being the easiest to implement as it works well when generating a grid-based maze.

MAZE SURVIVOR– Computer Science NEA

End User Survey:

To make sure my application appeals to people who play video games, I am going to need input from people who enjoy video games by asking them a series of questions about what they would want to see in my application.

After I have created my application, I will ask the same end user for feedback on my application for I when evaluate the application

Name of End User:	How big do you think the maze be?	What keys should be used to move the Runner and the Tagger
Zach	"I think the maze should be 100 x 100"	"If you're doing a two-player game, it might be best to have the Runner to move using WASD and have the Tagger move using IJKL"

Practical Limitations:

Ideally you could program the mazes to be very big, however there is the issue that making the maze too big could cause a hindrance to the applications' performance which would make the game unenjoyable for players.

Current system specifications:

Operating system:	Windows 11 Home
Computer model:	Lenovo Yoga Slim 6
RAM:	8GB (Speed: 5200 MT/s)
CPU:	13 th Gen Intel® Core™ i5-13500 H (2.60 GHz)
System Type:	64-bit operating system, x64-based processor
Programming language:	IDLE (Python 3.12 64 bit)

MAZE SURVIVOR– Computer Science NEA

Objectives:

- UI
 - Main Menu
 - A button to start the game
 - 3 buttons that set the games difficulty easy, medium, hard
 - Easy mode means the survival time is 1 minutes
 - Medium mode means the survival time is 2 minutes
 - Hard mode means is 3 minutes
 - A button to close the Main Menu
- Gameplay
 - Generate the maze when the button to start the game is pressed
 - Generate a maze in an array grid using a modified version Prim's algorithm to allow for gaps in the maze making sure the maze has no exits.
 - The Maze is then written into a text document
 - Tkinter then reads the text document and then draws the maze
 - Spawn in the player and the tagger at random starting points in the maze
 - The game ends once the timer has reached zero or the tagger reaches the player
 - Each second a method is run to see if the tagger position in the maze is the same as the Runner position
 - If the tagger gets the Runner, it displays a message saying "Tagger Wins!"
 - If the Runner survives display a message saying "Runner Wins!"
 - The game then gives the option to go back to the main menu or quit
- Tagger
 - Controls
 - Move up
 - Press the I key or the up arrow
 - Ensure that when pressing I key, the player will not move through walls
 - Move down
 - Press the K key or the up arrow
 - Ensure that when pressing K key, the player will not move through walls
 - Move right
 - Press the L key or the right arrow
 - Ensure that when pressing L key, the player will not move through walls
 - Move left
 - Press the J key or the left arrow
 - Ensure that when pressing J key, the player will not move through walls

MAZE SURVIVOR– Computer Science NEA

- Runner
 - Controls
 - Move up
 - Press the W key or the up arrow
 - Ensure that when pressing W key, the player will not move through walls
 - Move down
 - Press the S key or the up arrow
 - Ensure that when pressing S key, the player will not move through walls
 - Move right
 - Press the D key or the right arrow
 - Ensure that when pressing D key, the player will not move through walls
 - Move left
 - Press the A key or the left arrow
 - Ensure that when pressing A key, the player will not move through walls

MAZE SURVIVOR– Computer Science NEA

Design

Project Structure:

The main project will be broken into 3 python files all stored in one folder MazeGeneration.py DrawMaze.py and Main.py

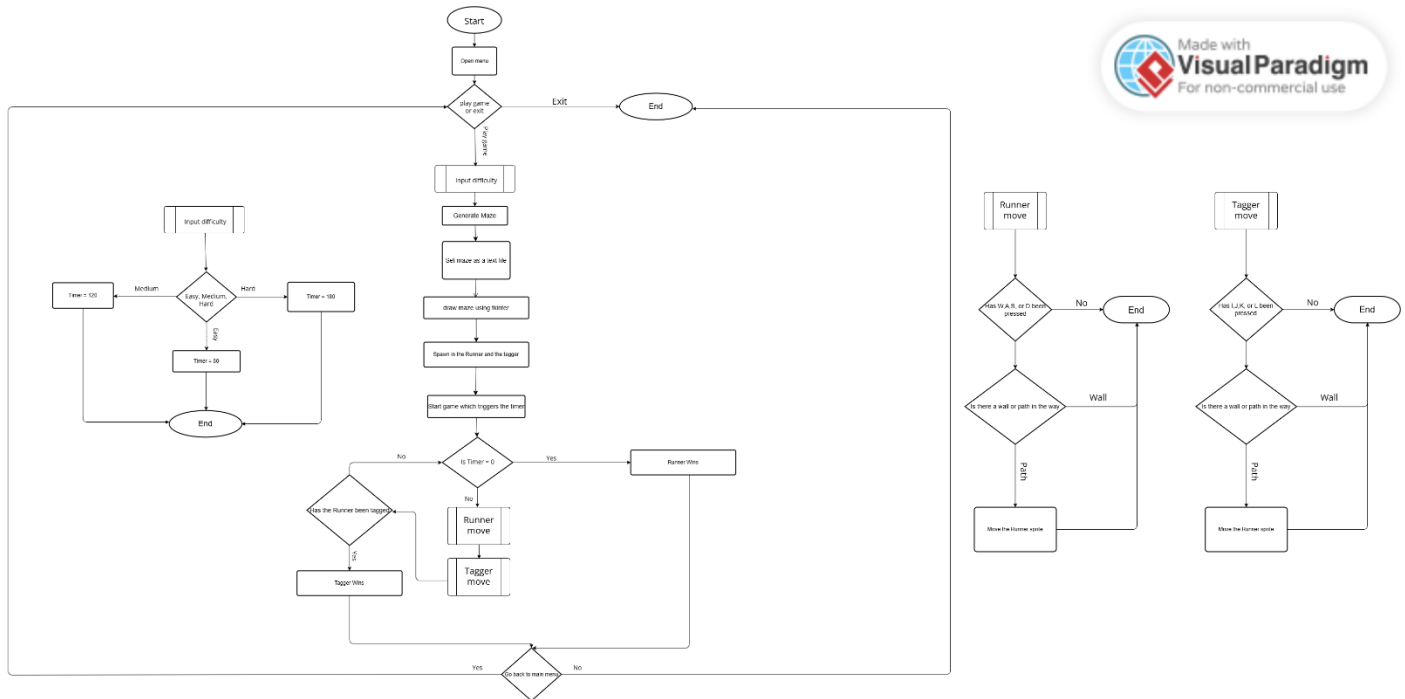
MazeGeneration.py will contain the program used to generate the maze by inheriting the random library and using Prim's algorithm. It will generate a 2d array which will represent a maze.

DrawMaze.py will grab the 2d array from the MazeGeneration.py and write it into a text file. By also inheriting the Tkinter library, it will read the text file and then draw the maze.

This Python will run the game, inheriting Tkinter to make a main menu, grabbing the generated maze using Tkinter from DrawMaze.py creating the Runner and Tagger sprites, creating the timer, and doing the win conditions checks to see if the Runner or Tagger has won the game.

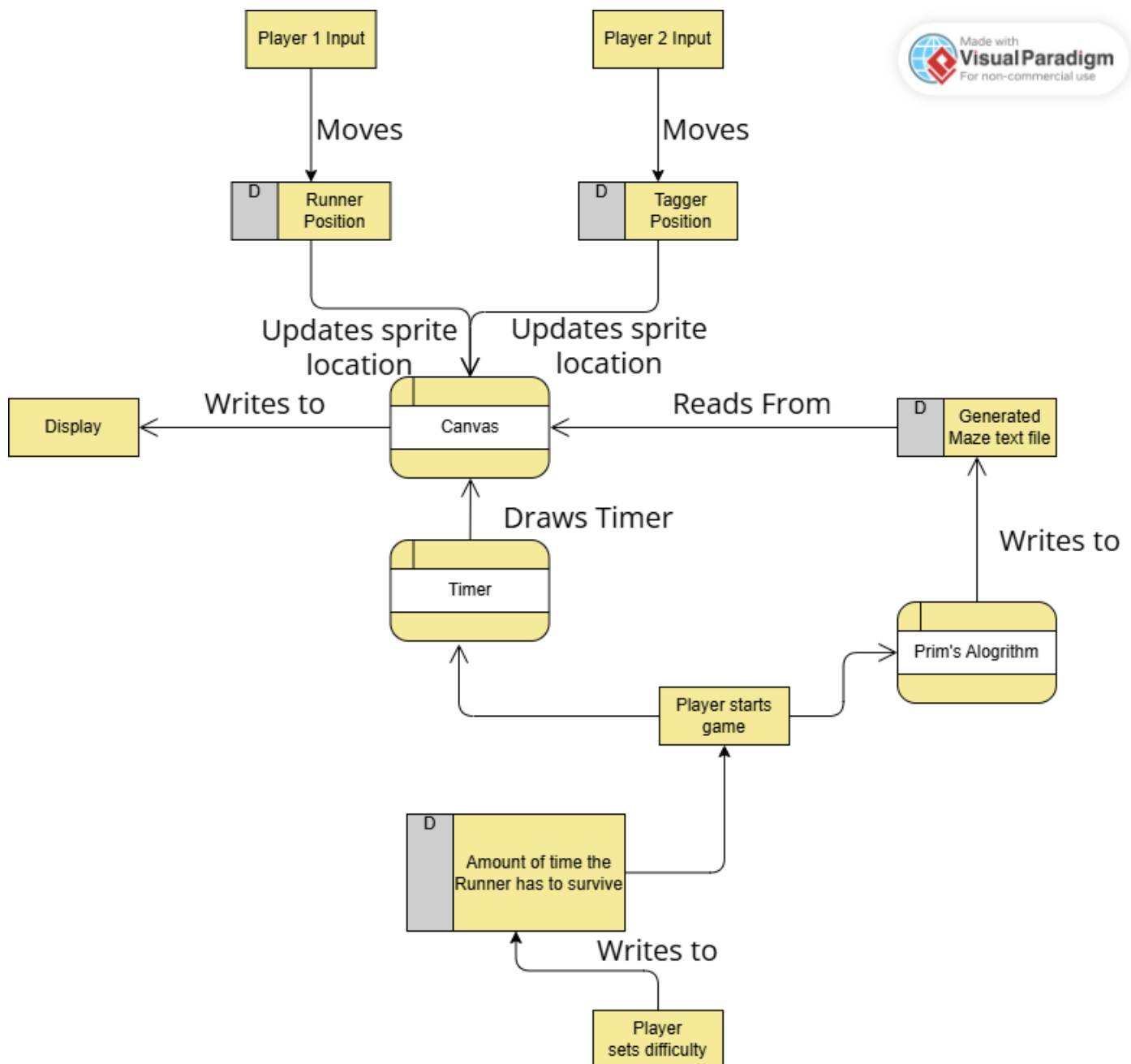
MAZE SURVIVOR– Computer Science NEA

Flowchart Diagram



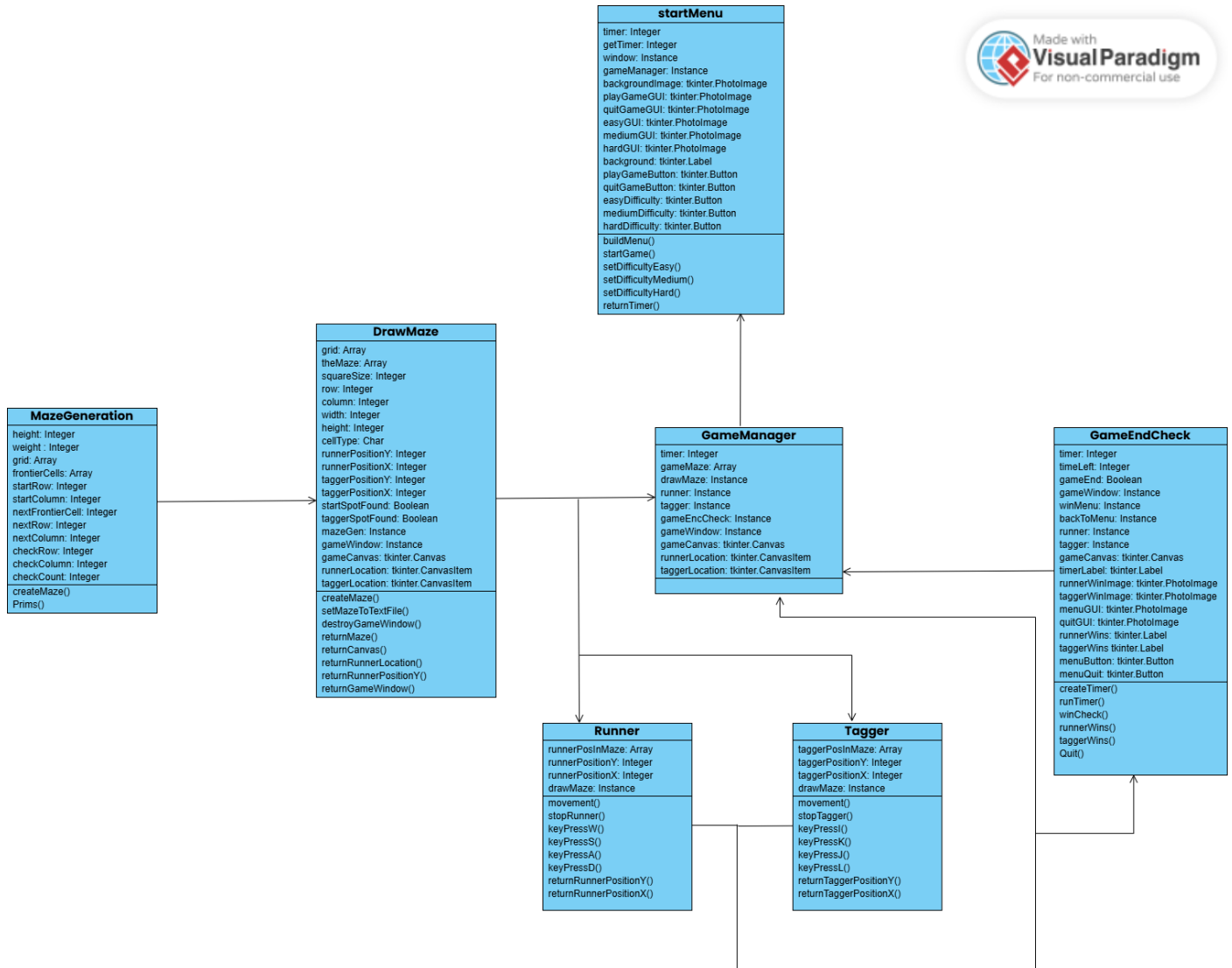
MAZE SURVIVOR– Computer Science NEA

Data Flow Diagram



MAZE SURVIVOR– Computer Science NEA

Class Diagram



MAZE SURVIVOR– Computer Science NEA

Class Structure:

MazeGeneration.py

Class: MazeGeneration

Methods of MazeGeneration:

Name	Method type	Parameters	Description
<code>_init_</code>	Constructor method	height weight grid	Initiates the class
<code>createMaze</code>	Implementation method	None	A Method that fills a 2d-array with all the barriers and walls
<code>Prims</code>	Implementation and Interface method (getter)	None	A method that applies Prim's algorithm to the maze which generates the paths that the Runner and Tagger can walk on

Attributes of MazeGeneration:

Name	Type	Initial value	Description
<code>height</code>	Integer	50	Constant which holds the height of the Maze
<code>weight</code>	Integer	50	Constant which holds the width of the maze
<code>grid</code>	Array	A 2d array	A 50x50 2d array with each cell being filled with "b" and "w"
<code>frontierCells</code>	Array	[]	Array which holds tuple pairs, with the pairs being the coordinates of the frontier cells in the maze.
<code>startRow</code>	Integer	Random number between 1 and 48	Holds the value y coordinate of the cell for where the maze generation starts
<code>startColumn</code>	Integer	Random number 2 and 48	Holds the value x coordinate of the cell for where the maze generation starts
<code>nextFrontierCell</code>	Integer	Random number between 1 and one less of the number items in <code>frontierCells</code>	Holds the value of which tuple pair will be picked in <code>FrontierCells</code> . This determines which cell is chose as the path
<code>nextRow</code>	Integer	Random Integer	First item in the tuple pair, being the Y coordinate of the next frontier cell

MAZE SURVIVOR– Computer Science NEA

nextColumn	Integer	Random Integer	Second item in the tuple pair, being the X coordinate of the next frontier cell
checkRow	Integer	Random Integer	First item in the tuple pair, being the Y coordinate of the frontier cell being checked
checkColumn	Integer	Random Integer	First item in the tuple pair, being the X coordinate of the frontier cell being checked
checkCount	Integer	0	A counter variable which increases by 1 each time a frontier cell has a path in front of it

MAZE SURVIVOR– Computer Science NEA

DrawMaze.py

Class: DrawMaze

Methods of DrawMaze:

Name	Method type	Parameters	Description
__init__	Constructor method	None	Initiates the class
SetMazeToTextFile	Implementation method	None	A method which takes the 2d array of the generated maze and turns it into a text file
createMaze	Implementation method	None	A method that takes the text file of the generated maze and then uses Tkinter to draw the maze on a Tkinter Canvas
destroyGameWindow	Implementation method	None	A method that destroys the window of the game being played
returnMaze	Interface method (getter)	None	A getter that returns the value of the 2d array of the generated maze
returnCanvas	Interface method (getter)	None	A getter that returns the Tkinter Canvas
returnRunnerLocation	Interface method (getter)	None	A getter that returns the runner sprite which is a Tkinter Canvas item
returnRunnerPositionY	Interface method (getter)	None	A getter that returns the y coordinate of the Runner's position in the Maze
returnRunnerPositionX	Interface method (getter)	None	A getter that returns the x coordinate of the Runner's position in the Maze
returnTaggerLocation	Interface method (getter)	None	A getter that returns the tagger sprite which is a Tkinter Canvas item
returnTaggerPositionY	Interface method (getter)	None	A getter that returns the y coordinate of the Tagger's position in the Maze

MAZE SURVIVOR– Computer Science NEA

returnTaggerPositionX	Interface method (getter)	None	A getter that returns the x coordinate of the Tagger's position in the Maze
returnGameWindow	Interface method (getter)	None	A getter method that returns instance of the class Tkinter

Attributes of DrawMaze class:

Name	Type	Initial value	Description
mazeGen	Instance	N/A	A memory location for the MazeGeneration class
grid	Array	A 2d array of the generated maze	A 50x50 2d array with each cell being filled with "b", "w" and "P"
theMaze	Array	[]	A 50x50 2d array with each cell being filled with "b", "w" and "P"
gameWindow	Instance	N/A	A memory location for the Tkinter class
gameCanvas	tkinter.Canvas		A memory location for the tkinter canvas to use to draw the maze
squareSize	Integer	15	A constant used to set the size of each cell when draw with Tkinter
row	Integer	50	Constant which holds the number of rows in the 2d-array
column	Integer	50	Constant which holds the columns in the 2d array
width	Integer	50* 15	Constant which holds the width of the Tkinter Canvas where the maze will be drawn
height	Integer	50 * 15	Constant which holds the height of the Tkinter Canvas where the Maze will be drawn
cellType	Char	Either "P" "w" or "b"	A condition variable which holds what type of cell is currently being looked at which determines the colour of that cell

MAZE SURVIVOR– Computer Science NEA

runnerLocation	tkinter.CanvasItem	N/A	A memory location for the runner sprite
taggerLocation	tkinter.CanvasItem	N/A	A memory location for tagger sprite
runnerPositionY	Integer	Random number between 0 and 49	Holds the value of the y-coordinate of where the runner is in the maze
runnerPositionX	Integer	Random number between 0 and 49	Holds the value of the x-coordinate of where the runner is in the maze
taggerPositionY	Integer	Random number between 0 and 49	Holds the value of the y-coordinate of where the runner is in the maze
taggerPositionX	Integer	Random number between 0 and 49	Holds the value of the x-coordinate of where the runner is in the maze
startSpotFound	Boolean	False	A flag which is set to true when the starting position for the runner has been found
taggerSpotFound	Boolean	False	A flag which is set to true when the starting position for the tagger has been found

MAZE SURVIVOR– Computer Science NEA

Main.py

Class: startMenu

Methods of startMenu

Name	Method Type	Parameters	Description
__init__	Constructor method	None	Initiates the class
buildMenu	Implementation method	None	A method that makes the main menu for the application using Tkinter
startGame	Implementation method	None	A method that runs the GameManager class when the Play Game button is pressed
quitGame	Implementation method	None	A method that shuts down the application when the quit button is pressed
setDifficultyEasy	Interface method (setter)	None	A method that sets the time that the runner has to survive to 60 seconds when the easy difficulty button is pressed
setDifficultyMedium	Interface method (setter)	None	A method that sets the time that the runner must survive to 120 seconds when the medium difficulty button is pressed
setDifficultyHard	Interface method (setter)	None	A method that sets the time that the runner must survive to 180 seconds when the hard difficulty button is pressed
returnTimer	Interface method (getter)	None	A getter that returns the value of the time the runner must survive for

MAZE SURVIVOR– Computer Science NEA

Attributes of startMenu class:

Name	Type	Initial value	Description
window	Instance	N/A	A memory location for the Tkinter class
gameManager	Instance	N/A	A memory location for the GameManager class
backgroundImage	tkinter.PhotoImage	menuBackground.png	A memory location for the image menuBackground.png
playGameGUI	tkinter.PhotoImage	playGameButton.png	A memory location for the image playGameButton.png
quitGameGUI	tkinter.PhotoImage	quitButton.png	A memory location for the image quitButton.png
easyGUI	tkinter.PhotoImage	easyButton.png	A memory location for the image easyButton.png
mediumGUI	tkinter.PhotoImage	mediumButton.png	A memory location for the image easyButton.png
hardGUI	tkinter.PhotoImage	hardButton.png	A memory location for the image hardButton.png
background	tkinter.Label	N/A	A memory location for the background of the main menu
playGameButton	tkinter.Button	N/A	A memory location for the Play game button which runs the startGame method when pressed
quitGameButton	tkinter.Button	N/A	A memory location for the Quit button which runs the quitGame method when pressed
easyDifficulty	tkinter.Button	N/A	A memory location for the Easy Difficulty button which runs the setDifficultyEasy method when pressed

MAZE SURVIVOR– Computer Science NEA

mediumDifficulty	tkinter.Button	N/A	A memory location for the Medium Difficulty Button which runs the setDifficultyMedium method when pressed
hardDifficulty	tkinter.Button	N/A	A memory location for the Medium Difficulty Button which runs the setDifficultyMedium method when pressed
timer	Integer	60	Holds the value of how in seconds the runner must survive. At first is set to 60 but changes as the game difficulty is changed
getTimer	Integer	Either 60, 120 or 180	Holds the final value of timer which is obtain by the returnTimer method

Class: GameManager

Methods of GameManager

Name	Method Type	Parameters	Description
__init__	Constructor method	Timer	Initiates the class

Attributes of GameManager class:

Name	Type	Initial value	Description
drawMaze	Instance	N/A	A memory location for the DrawMaze class
runner	Instance	N/A	A memory location for the Runner class
tagger	Instance	N/A	A memory location for the Tagger class
gameEndCheck	Instance	N/A	A memory location for the gameEndCheck class
gameWindow	Instance	N/A	A memory location for the Tkinter class
gameCanvas	tkinter.Canvas	N/A	A memory location for the tkinter canvas to use to draw the maze
runnerLocation	tkinter.CanvasItem	N/A	A memory location for the runner sprite

MAZE SURVIVOR– Computer Science NEA

taggerLocation	tkinter.CanvasItem	N/A	A memory location for the tagger sprite
gameMaze	Array	A 2d array of the generated maze	A 50x50 2d array with each cell being filled with “b”, “w” “P” and “runner” & “tagger”
timer	Integer	Either 60, 120 or 180	Holds the value of how in seconds the runner must survive. At first is set to 60 but changes as the game difficulty is changed

Class: Runner

Methods of Runner

Name	Method Type	Parameters	Description
__init__	Constructor method	gameMaze, gameCanvas, runnerLocation, drawMaze	Initiates the class
movement	Implementation method	None	A method that binds the WASD keys so that the runner sprite can move in the maze
stopRunner	Implementation method	None	A method that unbinds the WASD keys when the game is finished
keyPressW	Implementation method	event	A method that is run when the “W”-key is pressed. This causes the runner sprite to move up in the maze
keyPressS	Implementation method	event	A method that is run when the “S”-key is pressed. This causes the runner sprite to move down in the maze
keyPressA	Implementation method	event	A method that is run when the “A”-key is pressed. This causes the runner sprite to move left in the maze

MAZE SURVIVOR– Computer Science NEA

keyPressD	Implementation method	event	A method that is run when the “D”-key is pressed. This causes the runner sprite to move right in the maze
returnRunnerPositionY	Interface method (getter)	None	A getter that returns the y coordinate of the Runner’s position in the Maze
returnRunnerPositionX	Interface method (getter)	None	A getter that returns the x coordinate of the Runner’s position in the Maze

Attributes of Runner class:

Name	Type	Initial value	Description
drawMaze	Instance	N/A	A memory location for the DrawMaze class
gameCanvas	tkinter.Canvas	N/A	A memory location for the tkinter canvas to use to draw the maze
runnerLocation	tkinter.CanvasItem	N/A	A memory location for the runner sprite
runnerPosInMaze	Array	A 2d array of the generated maze	A 2d array that holds the maze that keeps track of where the runner is in the maze
runnerPositionY	Integer	A number between 0-49	Holds the Y-coordinate of the runner’s starting position
runnerPositionX	Integer	A number between 0-49	Holds the X-coordinate of the runner’s starting position

MAZE SURVIVOR– Computer Science NEA

Class: Tagger

Methods of Tagger

Name	Method Type	Parameters	Description
__init__	Constructor method	gameMaze, gameCanvas, runnerLocation, drawMaze	Initiates the class
movement	Implementation method	None	A method that binds the IJKL keys so that the tagger sprite can move in the maze
stopTagger	Implementation method	None	A method that unbinds the IJKL keys when the game is finished
keyPressI	Implementation method	event	A method that is run when the “I”-key is pressed. This causes the tagger sprite to move up in the maze
keyPressK	Implementation method	event	A method that is run when the “K”-key is pressed. This causes the tagger sprite to move down in the maze
keyPressJ	Implementation method	event	A method that is run when the “J”-key is pressed. This causes the tagger sprite to move do in the left maze
keyPressL	Implementation method	event	A method that is run when the “L”-key is pressed. This causes the tagger sprite to move right in the maze
returnTaggerPositionY	Interface method (getter)	None	Holds the value of the y-coordinate of where the runner is in the maze
returnTaggerPositionX	Interface method (getter)	None	A getter that returns the x-coordinate of the

MAZE SURVIVOR– Computer Science NEA

			Tagger's position in the Maze
--	--	--	-------------------------------

Attributes of Tagger class:

Name	Type	Initial value	Description
drawMaze	Instance	N/A	A memory location for the DrawMaze class
gameCanvas	tkinter.Canvas	N/A	A memory location for the tkinter canvas to use to draw the maze
taggerLocation	tkinter.CanvasItem	N/A	A memory location for the tagger sprite
taggerPosInMaze	Array	A 2d array of the generated maze	A 2d array that holds the maze that keeps track of where the tagger is in the maze
taggerPositionY	Integer	A number between 0-49	Holds the value of the y-coordinate of where the tagger is in the maze
taggerPositionX	Integer	A number between 0-49	Holds the value of the x-coordinate of where the tagger is in the maze

Class: GameEndCheck

Methods of GameEndCheck

Name	Method Type	Parameters	Description
__init__	Constructor method	gameWindow gameCanvas, timer, runner, tagger	Initiates the class
createTimer	Implementation method	None	A method that builds the Tkinter label which displays the timer
runTimer	Implementation method	timeleft	A method that runs recursively which updates the timer and runs the method win check. The method stops calling itself once time left is less than 0
winCheck	Implementation method	None	A method that checks to see if the tagger's position is

MAZE SURVIVOR– Computer Science NEA

			equal to the runner's position
runnerWins	Implementation method	None	A method that runs if the timer is less than 0. This then builds the Runner wins menu using Tkinter
taggerWins	Implementation method	None	A method that runs when the tagger position is equal to the runner position. This then builds the Tagger wins menu using Tkinter
backToMenu	Implementation method	None	A method that triggers the initiation of the startMenu class when the Menu button is pressed in either the Tagger wins menu or Runner wins menu
Quit	Implementation method	None	A method that shuts down the application when the Quit button is pressed in either the Tagger wins menu or Runner wins menu

Attributes of GameEndCheck class:

Name	Type	Initial value	Description
gameWindow	Instance	N/A	A memory location for the Tkinter class for the game application
winMenu	Instance	N/A	A memory location for the Tkinter class for the winner menu application
backToMenu	Instance	N/A	A memory location for the StartMenu class

MAZE SURVIVOR– Computer Science NEA

gameCanvas	tkinter.Canvas	N/A	A memory location the Tkinter canvas for the game application
runner	Instance	N/A	A memory location for the Runner class
tagger	Instance	N/A	A memory location for the Tagger class
timer	Integer	Either 60, 120 or 180	Holds the value of how in seconds the runner must survive.
timeLeft	Integer	Whatever the initial value of timer is	Local variable which holds the runTimer value of timer and decreases by 1 per second.
gameEnd	Boolean	False	A flag which indicates wherever the game is over. It is set to True when
timerLabel	tkinter.Label	N/A	A memory location for the label which displays the amount of time left the runner has to survive
runnerWinImage	tkinter.PhotoImage	runnerWins.png	A memory location for the image runnerWins.png
taggerWinImage	tkinter.PhotoImage	taggerWins.png	A memory location for the image taggerWins.png
menuGUI	tkinter.PhotoImage	menuButton.png	A memory location for the image menuButton.png
quitGUI	tkinter.PhotoImage	quitButton.png	A memory location for the image quitButton.png
runnerWins	tkinter.Label	N/A	A memory location for the background of the Runner wins menu
taggerWins	tkinter.Label	N/A	A memory location for the background of the Tagger wins menu

MAZE SURVIVOR– Computer Science NEA

menuButton	tkinter.Button	N/A	A memory location for the Menu button which runs the backToMenu method when pressed
menuQuit	tkinter.Button	N/A	A memory location for the Quit button which runs the Quit method when pressed

Data Structures:

Data Structure- self.grid

In my application, the maze will be generated in a 2-d grid which indicates where the path, wall and barriers in the maze.

Here is a rough idea of what the grid will look like.

	0	1	2	3	4
0	barrier	barrier	barrier	barrier	barrier
1	barrier	barrier	barrier	barrier	barrier
2	barrier	barrier	wall	path	wall
3	barrier	barrier	wall	path	wall
4	barrier	barrier	wall	path	wall

MAZE SURVIVOR– Computer Science NEA

Algorithms:

Prim's algorithm

To help me write out the algorithm in python for generating the maze, I used the following pseudocode to assist me

```
function Prim(vertices, edges) is
    for each vertex in vertices do
        cheapestCost[vertex] ← ∞
        cheapestEdge[vertex] ← null

    explored ← empty set
    unexplored ← set containing all vertices

    startVertex ← any element of vertices
    cheapestCost[startVertex] ← 0

    while unexplored is not empty do
        // Select vertex in unexplored with minimum cost
        currentVertex ← vertex in unexplored with
minimum    cheapestCost[vertex]
        unexplored.remove(currentVertex)
        explored.add(currentVertex)

        for each edge (currentVertex, neighbor) in edges
do
            if neighbor in unexplored and
weight(currentVertex, neighbor) < cheapestCost[neighbor]
THEN
                cheapestCost[neighbor] ←
weight(currentVertex, neighbor)
                cheapestEdge[neighbor] ← (currentVertex,
neighbor)

    resultEdges ← empty list
    for each vertex in vertices do
        if cheapestEdge[vertex] ≠ null THEN
            resultEdges.append(cheapestEdge[vertex])

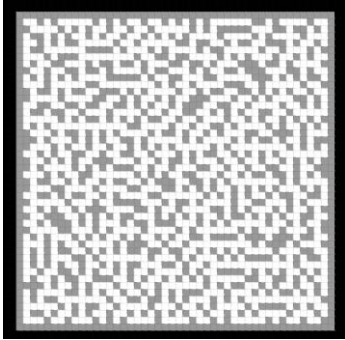
    return resultEdges
```

(Source: https://en.wikipedia.org/wiki/Prim%27s_algorithm)

MAZE SURVIVOR– Computer Science NEA

Graphical user interface:

Here are a series of concepts of the GUI for Maze Survivor



Maze GUI



Tagger



Runner



Timer

MAZE SUVIVOR

PLAY GAME

QUIT

Difficulty:

EASY

MEDIUM

HARD

Main
Menu

RUNNER WINS!

MENU

QUIT

Menu for when the
runner wins

TAGGER WINS!

MENU

QUIT

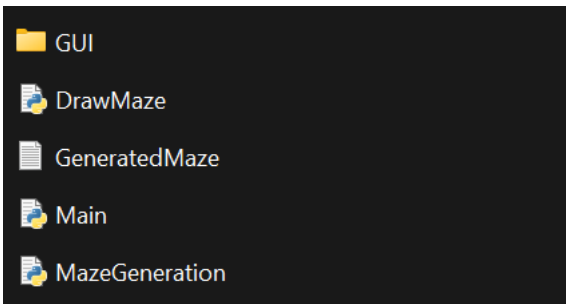
Menu for when the
tagger wins

MAZE SURVIVOR– Computer Science NEA

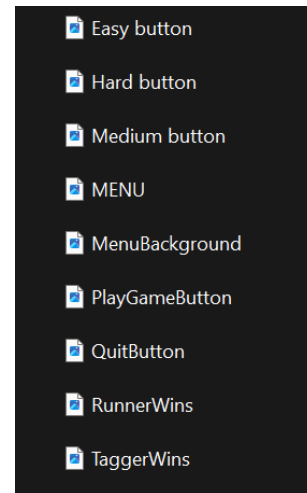
Technical Solution:

File structure

Root



GUI



MAZE SURVIVOR– Computer Science NEA

Code

MazeGeneration.py

```
#MazeGeneration.py
```

```
import random
```

```
#Inniates the class MazeGeneration, this class will be used to generate the maze
```

```
class MazeGeneration:
```

```
    #Defining the attributes of the Maze generation object
```

```
    def __init__(self,height, width, grid =None):
```

```
        self.height = height
```

```
        self.width = width
```

```
        #grid is the where the maze data is stored in the 2d array, this is modified
```

```
        self.grid = grid
```

```
#The method which creates a 2D-array which will be the grid that will be put into the Maze generation algorithm
```

```
    def createMaze(self):
```

```
        #Sets all of the cells in the grid to barriers
```

```
        self.grid = [["b" for x in range(self.width)] for y in range(self.height)]
```

```
        for y in range(2,self.height-2):
```

```
            for x in range(2,self.width-2):
```

```
                #sets everything but the last two layers of cells to a wall
```

```
                ("w")
```

```
                self.grid[y][x] = "w"
```

```
#The method which generates a maze using prims algorithm
```

```
    def Prims(self):
```

```
        self.createMaze()
```

```
        #An array of the coordinates of the possible cells that could be turned into paths, Tuples are used to store the coordinates as individual items
```

```
        frontierCells = []
```

```
        #Picks the a random Y coordinates within the confines of the walls
```

```
        ("w")
```

```
        StartRow = random.randint(2,48)
```

```
        #Picks the a random X coordinates within the confines of the walls
```

```
        ("w")
```

```
        StartCollumn = random.randint(2,48)
```

MAZE SURVIVOR- Computer Science NEA

```
#Sets the randomly pick coordinates as a path ("P")
self.grid[StartRow][StartCollumn] = "P"

#From the starting point the cells around it are selected as frontier
cells

#Start of the first frontier cell check
#Looks at west of the starting point by 1 cell
#if the cell it's looking at is wall ("w") and not a barrier ("b") then
that cell's coordinates get appended a tuple in frontierCells
    if self.grid[StartRow-1][StartCollumn] != "b" and self.grid[StartRow-
1][StartCollumn] == "w":
        frontierCells.append((StartRow-1,StartCollumn))

#Looks east of the starting point by 1 cell
#if the cell it's looking at is wall ("w") and not a barrier ("b") then
that cell's coordinates get appended as a tuple in frontierCells
    if self.grid[StartRow+1][StartCollumn] != "b" and
self.grid[StartRow+1][StartCollumn] == "w":
        frontierCells.append((StartRow+1,StartCollumn))

# Looks south of the starting point by 1 cell
#if the cell it's looking at is wall ("w") and not a barrier ("b") then
that cell's coordinates get appended as a tuple in frontierCells
    if self.grid[StartRow][StartCollumn-1] != "b" and
self.grid[StartRow][StartCollumn-1] == "w":
        frontierCells.append((StartRow,StartCollumn-1))

#Looks north of the starting point by 1 cell
#if the cell it's looking at is wall ("w") and not a barrier ("b") then
that cell's coordinates get appended as a tuple in frontierCells
    if self.grid[StartRow][StartCollumn+1] != "b" and
self.grid[StartRow][StartCollumn+1] == "w":
        frontierCells.append((StartRow,StartCollumn+1))

#This WHILE loop will run until the maze has been fully generated
while len(frontierCells) > 0:

    #The next frontier cell is randomly selected from frontierCell, the
tuple pair of cooridnates gets stored in nextFrontierCell
    nextFrontierCell = random.randint(0,len(frontierCells)-1)

    #first item in the tuple pair (the Y cooridnate) is stored in
NextRow
    NextRow = frontierCells[nextFrontierCell][0]
    #second item in the tuple pair (the X cooridnate) is stored in
NextCollumn
    NextCollumn = frontierCells[nextFrontierCell][1]
```

MAZE SURVIVOR- Computer Science NEA

```
#Sets the chosen frontier cell to a path ("P")
self.grid[NextRow][NextCollumn] = "P"

#removes the frontier cell from the array which was chosen to be
the next path cell
frontierCells.pop(nextFrontierCell)

#The cells around the chosen path cell are checked for new frontier
cells to be added to the array frontierCells

#Start of the frontier cell check
#Looks west if the chosen path cell by 1 cell
#if the cell it's looking at is wall("w") and not a path ("P") then
that cell's coordinates get appended as a tuple in frontierCells
if self.grid[NextRow-1][NextCollumn] != "P" and self.grid[NextRow-
1][NextCollumn] == "w":
    frontierCells.append((NextRow-1,NextCollumn))

#Looks east of the chosen path cell by 1 cell
#if the cell it's looking at is wall ("w") and not a path ("P")then
that cell's coordinates get appended as a tuple in frontierCells
if self.grid[NextRow+1][NextCollumn] != "P" and
self.grid[NextRow+1][NextCollumn] == "w":
    frontierCells.append((NextRow+1,NextCollumn))

#Looks south of the chosen path cellby 1 cell
#if the cell it's looking at is wall ("w") and not a path ("P")
then that cell's coordinates get appended as a tuple in frontierCells
if self.grid[NextRow][NextCollumn-1] != "P" and
self.grid[NextRow][NextCollumn-1] == "w":
    frontierCells.append((NextRow,NextCollumn-1))

#Looks north of the chosen path cell by 1 cell
#if the cell it's looking at is wall ("w") and not a path ("P")
then that cell's coordinates get appended as a tuple in frontierCells
if self.grid[NextRow][NextCollumn+1] != "P" and
self.grid[NextRow][NextCollumn+1] == "w":
    frontierCells.append((NextRow,NextCollumn+1))
#End of the frontier cell check

#This FOR loop will prune any frointer cells with a check count
greater than two
#The CheckCount is increased by one each time the frontier cell has
a path or barrier infront of it.
#The FOR loop iterates backwards to avoid an index error
for i in range(len(frontierCells)-1,-1,-1):

    #first item in the tuple pair (the Y coordidnate) from the
frontier cell being checked is stored CheckRow
    CheckRow = frontierCells[i][0]
```

MAZE SURVIVOR– Computer Science NEA

```
#second item in the tuple pair (the x coordinate) from the
frontier cell being checked is stored CheckCollumn
CheckCollumn = frontierCells[i][1]

CheckCount = 0

#Start of the prune check
#Looks west of the frontier cell being checked by 1 cell
if self.grid[CheckRow-1][CheckCollumn] == "b" or
self.grid[CheckRow-1][CheckCollumn] == "P":
    CheckCount = CheckCount + 1

#Looks east of the frontier cell being checked by 1 cell
if self.grid[CheckRow+1][CheckCollumn] == "b" or
self.grid[CheckRow+1][CheckCollumn] == "P":
    CheckCount = CheckCount + 1

#Looks south of the frontier cell being checked by 1 cell
if self.grid[CheckRow][CheckCollumn-1] == "b" or
self.grid[CheckRow][CheckCollumn-1] == "P":
    CheckCount = CheckCount + 1

#Looks north of the frontier cell being checked by 1 cell
if self.grid[CheckRow][CheckCollumn+1] == "b" or
self.grid[CheckRow][CheckCollumn+1] == "P":
    CheckCount = CheckCount + 1
#End of the prune check

#The frontier cell is pruned in which it's tuple pair of
coordinates are removed from the array
if CheckCount >= 2:
    frontierCells.pop(i)

#returns the 2d array stored in grid which will be used to draw out the
maze using Tkinter in DrawMaze.py
return self.grid
```

MAZE SURVIVOR– Computer Science NEA

DrawMaze.py

```
#DrawMaze.py

#Importing libraries
import sys
import tkinter as tk
import random
from tkinter import *

#File path of MazeGeneration.py
sys.path.insert(1, "C:\\PROJECT\\MazeGeneration.py")

#imports the MazeGeneration.py from it being imported from the sys.path
function
import MazeGeneration

#Class which will be used to draw the Maze using Tkinter
class DrawMaze():
    def __init__(self):
        #Initiates the class by running the methods SetMazeToTextFile and
        createMaze
        self.SetMazeToTextFile()
        self.createMaze()

#The method which processes the Generated Maze into a text file so it can be
drawn out with Tkinter
    def SetMazeToTextFile(self):
        #Declares mazeGen as the memory location for the MazeGeneration class,
        50 is entered as the height and width parameters to set the maze size to 50x50
        mazeGen = MazeGeneration.MazeGeneration(50,50)

        #grid stores the array of the maze which has been generated
        grid = mazeGen.Prims()

        #Creates a text file called GeneratedMaze.txt which contains the array
        of the maze generated from MazeGeneration.py
        with open('GeneratedMaze.txt' , 'w') as F:
            for y in range (len(grid)):
                for x in range (len(grid[0])):
                    F.write(str(grid[y][x]) + ' ')
                F.write("\n")
            F.close()

#The Method which draws the Maze using Tkinter, while also making the game
window
    def createMaze(self):
```

MAZE SURVIVOR- Computer Science NEA

```
#Empty Array which will store the contents of GeneratedMaze.txt when it
as been read
self.theMaze = []
#Creating the game window using Tkinter, while also setting the window
name and it's dimensions
self.gameWindow = tk.Tk()
self.gameWindow.title('Maze Survivor')
self.gameWindow.geometry("1280x800")

#A constant which is used as a scale when setting up the the height and
width of each cell in the maze
SquareSize = 15

#Opens the text file GeneratedMaze.txt
with open('GeneratedMaze.txt' , 'r') as F:
    #for each line in GeneratedMaze.txt it is appended into
self.theMaze
    for lines in F.readlines():
        self.theMaze.append(lines.split())
F.close()

#Variables storing how many rows and collumns are in the 2D-array of
self.theMaze
row = len(self.theMaze)
column = len(self.theMaze[0])

#The height and width of the maze is determined by multiplying row and
column by squareSize
Width = row * SquareSize
Height = column * SquareSize

#the self.GameCanvas is created using the Canvas Tkinter Class, this
creates an area in the gamewindow for the maze to be drawn
self.gameCanvas = Canvas(self.gameWindow, width = Width , height =
Height)
self.gameCanvas.pack()

#Nested FOR loop to draw the maze
for y in range (len(self.theMaze)):
    for x in range (len(self.theMaze[0])):

        #Goes through every cell in the Maze
        cellType = self.theMaze[y][x]

        #These variables are used as parameters for the
create_rectangle Tkinter function so that each cell is drawn as a square
        #The dimensions of each square are made constant by having the
y and x coordinate of the cell being multiplyed by squareSize
        #The value of squareSize is then added on to the respective
variables which is stored in a different variables.... (NEED HELP TO EXPLAIN
THIS BIT)
```

MAZE SURVIVOR– Computer Science NEA

```

yCoordinate1 = y * SquareSize
yCoordinate2 = yCoordinate1 + SquareSize
xCoordinate1 = x * SquareSize
xCoordinate2 = xCoordinate1 + SquareSize

#If the current cell in the Maze is a path, then the
create_rectangle function will create a green square in the self.GameCanvas to
represent a path
    if cellType == 'P':
        self.gameCanvas.create_rectangle(xCoordinate1, yCoordinate1,
xCoordinate2, yCoordinate2, fill = "green", outline = "green")

#If the current cell in the Maze is a wall or barrier, then the
create_rectangle function will create a black square self.GameCanvas to
represent a both
    elif cellType == 'b' or cellType == 'w':
        self.gameCanvas.create_rectangle(xCoordinate1,
yCoordinate1, xCoordinate2, yCoordinate2, fill = "black", outline = "black")

#WHILE loop to to find a location for the Runner in the maze
StartSpotFound = False
while StartSpotFound == False:
    #Randomly picks a row and collumn in the maze
    self.runnerPositionY = random.randint(0, (row-1))
    self.runnerPositionX = random.randint(0, (column-1))

    #If the coordinate is equal to a path then set that cell to Runner
    if self.theMaze[self.runnerPositionY][self.runnerPositionX] == 'P':
        self.theMaze[self.runnerPositionY][self.runnerPositionX] =
'Runner'

        StartSpotFound = True

#More variables which are the parameters for the create_rectangle
Tkinter function so that the Runner sprite can be drawn
startSpotYCoordinate1 = self.runnerPositionY * SquareSize
startSpotYCoordinate2 = startSpotYCoordinate1 + SquareSize
startSpotXCoordinate1 = self.runnerPositionX * SquareSize
startSpotXCoordinate2 = startSpotXCoordinate1 + SquareSize

#The attribute self.runnerLocation which stores the Runner sprite,
which is created as blue square on the self.GameCanvas to represent the runner
self.runnerLocation =
self.gameCanvas.create_rectangle(startSpotXCoordinate1, startSpotYCoordinate1,
startSpotXCoordinate2, startSpotYCoordinate2, fill = "blue", outline = "blue")

#WHILE loop to to find a location for the Tagger in the Maze
TaggerSpotFound = False
while TaggerSpotFound == False:
    #Randomly picks a row and collumn in the maze
    self.taggerPositionY = random.randint(0, (row-1))
```

MAZE SURVIVOR– Computer Science NEA

```
self.taggerPositionX = random.randint(0, (column-1))

#If the coordinate is equal to a path then set that cell to tagger
if self.theMaze[self.taggerPositionY][self.taggerPositionX] == 'P':
    self.theMaze[self.taggerPositionY][self.taggerPositionX] =
'tagger'

    TaggerSpotFound = True

#More variables which are the parameters for the create_rectangle
Tkinter function so that the Runner sprite can be drawn
taggerSpotYCoordinate1 = self.taggerPositionY * SquareSize
taggerSpotYCoordinate2 = taggerSpotYCoordinate1 + SquareSize
taggerSpotXCoordinate1 = self.taggerPositionX * SquareSize
taggerSpotXCoordinate2 = taggerSpotXCoordinate1 + SquareSize

#The attribute self.taggerLocation which stores the tagger sprite,
which is created as red square on the self.GameCanvas to represent the tagger
self.taggerLocation =
self.gameCanvas.create_rectangle(taggerSpotXCoordinate1,
taggerSpotYCoordinate1, taggerSpotXCoordinate2, taggerSpotYCoordinate2, fill =
"Red", outline = "Red")

#Method that returns the 2D-array of the Maze in its current state
def returnMaze(self):
    return self.theMaze

#Method that returns the GameCanvas in its current state
def returnCanvas(self):
    return self.gameCanvas

#Method that returns the attribute that holds the runner sprite
def returnRunnerLocation(self):
    return self.runnerLocation

#Method that returns the runner's position in the Y-axis of the maze (Row
of the 2D-array)
def returnRunnerPositionY(self):
    return self.runnerPositionY

#Method that returns the runner's position in the X-axis of the maze
(Column of the 2D-array)
def returnRunnerPositionX(self):
    return self.runnerPositionX

#Method that returns the attribute that holds the tagger sprite
def returnTaggerLocation(self):
    return self.taggerLocation

#Method that returns the tagger's position in the Y-axis of the maze (Row
of the 2D-array)
def returnTaggerPositionY(self):
```

MAZE SURVIVOR– Computer Science NEA

```
return self.taggerPositionY
```

```
#Method that returns the tagger's position in the X-axis of the maze  
(Column of the 2D-array)
```

```
def returnTaggerPositionX(self):  
    return self.taggerPositionX
```

```
#Method that destroys the game window for when the game is over
```

```
def destroyGameWindow(self):  
    self.gameWindow.destroy()
```

```
#Method that returns the game window in it's current state
```

```
def returnGameWindow(self):  
    return self.gameWindow
```

MAZE SURVIVOR– Computer Science NEA

```
#Main.py

#Importing libraries
import sys
import random
import tkinter as tk
from tkinter import *

#File path of DrawMaze.py
sys.path.insert(1, "C:\\PROJECT\\DrawMaze.py")

#Imports the DrawMaze.py from it being imported from the sys.path function
import DrawMaze

#Class which makes the main menu using Tkinter
class startMenu ():
    def __init__(self):
        #Initiates the class by setting the attribute self.timer to 60 and running
        the method buildMenu
        self.timer = 60
        self.buildMenu()

#Method that builds the main menu using Tkinter
def buildMenu(self):
    #Creating the main menu window using Tkinter, while also setting the
    window name and it's dimensions
    self.window = tk.Tk()
    self.window.title('Maze Survivor')
    self.window.geometry("1280x720")

    #Setting up the background of main menu
    #Using the PhotoImage function MenuBackground.png is stored in
    background
    background = PhotoImage(file = "C:\\PROJECT\\GUI\\MenuBackground.png")

    #Using the Label and .place functions, the bg label is created which is
    the image stored in background displayed in the window of the main menu
    bg = Label(self.window, image = background)
    bg.place(x = 0, y = 0)

    #Setting up the Play Game button
    #Using the PhotoImage function PlayGameButton.png is stored in
    PlayGameGUI
    PlayGameGUI = PhotoImage(file = "C:\\PROJECT\\GUI\\PlayGameButton.png")
```

MAZE SURVIVOR– Computer Science NEA

```
#Using the Button and .place functions the Play Game button is created
which is displayed in the window of the main menu
#The image of the button is the image stored in PlayGameGUI
#When the button is pressed it runs the method startGame
self.PlayGameButton = Button(self.window, image = PlayGameGUI, command
= self.startGame)
self.PlayGameButton.place(x = 50, y = 175)

#Setting up the Quit Game button
#Using the PhotoImage function QuitButton.png is stored in QuitGameGUI
QuitGameGUI = PhotoImage(file = "C:\\PROJECT\\GUI\\QuitButton.png")

#Using the Button and .place functions the Quit Game button is created
which is displayed in the window of the main menu
#The image of the button is the image stored in QuitGameGUI
#When the button is pressed it runs the method quitGame
self.QuitGameButton = Button(self.window, image = QuitGameGUI, command
= self.quitGame)
self.QuitGameButton.place(x = 730, y = 175)

#Setting up the Easy Difficulty button
#Using the PhotoImage function Easy button.png is stored in EasyGUI
EasyGUI = PhotoImage(file = "C:\\PROJECT\\GUI\\Easy button.png")

#Using the Button and .place functions the Easy Difficulty button is
created which is displayed in the window of the main menu
#The image of the button is the image stored in EasyGUI
#When the button is pressed it runs the method setDifficultyEasy
EasyButton = Button(self.window, image = EasyGUI, command =
self.setDifficultyEasy)
EasyButton.place(x = 400, y = 600)

#Setting up the Medium Difficulty button
#Using the PhotoImage function Medium button.png is stored in MediumGUI
MediumGUI = PhotoImage(file = "C:\\PROJECT\\GUI\\Medium button.png")

#Using the Button and .place functions the Medium Difficulty button is
created which is displayed in the window of the main menu
#The image of the button is the image stored in MediumGUI
#When the button is pressed it runs the method setDifficultyMedium
MediumButton = Button(self.window, image = MediumGUI, command =
self.setDifficultyMedium)
MediumButton.place(x = 600, y = 600)

#Setting up the Hard Difficulty button
#Using the PhotoImage function Hard button.png is stored in HardGUI
HardGUI = PhotoImage(file = "C:\\PROJECT\\GUI\\Hard button.png")

#Using the Button and .place functions the Hard Difficulty button is
created which is displayed in the window of the main menu
```

MAZE SURVIVOR- Computer Science NEA

```
#The image of the button is the image stored in HardGUI
#When the button is pressed it runs the method setDifficultyHard
HardButton = Button(self.window, image = HardGUI, command =
self.setDifficultyHard )
HardButton.place(x = 800, y = 600)

#Tkinter function that runs the while loop to keep the main menu window
responsive
self.window.mainloop()

#Method that destroys the main window menu if the Quit Game button is
pressed using the .destroy function
def quitGame(self):
    self.window.destroy()

#Method that updates the timer attribute to be 60 seconds
def setDifficultyEasy(self):
    self.timer = 60
    print("Timer now equals 60")

#Method that updates the timer attribute to be 120 seconds
def setDifficultyMedium(self):
    self.timer = 120
    print("Timer now equals 120")

#Method that updates the timer attribute to be 180 seconds
def setDifficultyHard(self):
    self.timer = 180
    print("Timer now equals 180")

#Method returns the attribute timer
def returnTimer(self):
    return self.timer

#Method that starts the game if the Play Game button is pressed
def startGame(self):
    #Destroys the main window menu
    self.window.destroy()

    #attribute that stores the value of the timer by running the method
return Timer
    self.GetTimer = self.returnTimer()

#Declares gameManger as the memory location for the GameManger Class
#The GameManager Class is then initiated with GetTimer as one of it's
parameters
gameManager = GameManager(self.GetTimer)
```

MAZE SURVIVOR– Computer Science NEA

```
#Class which sets up and runs the game
class GameManager():
    def __init__(self,timer):
        #Initiates the class, which starts the game.

        #Creates the attribute timer, which holds the value stored in the timer
parameter
        #The timer parameter will hold the value of GetTimer from the startMenu
class when the program is executed
        self.timer = timer

        #Declares DrawMaze as the memory location for the DrawMaze Class from
DrawMaze.py, Initiating the class
        drawMaze = DrawMaze.DrawMaze()

        #Sets up the parameters to be used in the various classes by using
getter methods from the DrawMaze class

        #Stores the self.theMaze attribute from the DrawMaze class into
gameMaze
        gameMaze = drawMaze.returnMaze()

        #Gets the self.theMaze attribute from the DrawMaze class and stores it
into gameMaze
        gameWindow = drawMaze.returnGameWindow()

        #Gets the self.gameCanvas attribute from the DrawMaze class and stores
it into gameCanvas
        gameCanvas = drawMaze.returnCanvas()

        #Gets the self.runnerLocation attribute from the DrawMaze class and
stores it into runnerLocation so it contains the runner sprite
        runnerLocation = drawMaze.returnRunnerLocation()

        #Gets the self.taggerLocation attribute from the DrawMaze class and
stores it into runnerLocation so it contains the tagger sprite
        taggerLocation = drawMaze.returnTaggerLocation()

        #Declares runner as the memory location for the Runner class
        #Initiating the class with the parameters gameMaze, GameCanvas and
runnerLocation so the Runner's position in the maze can be updated when it
moves
        #drawMaze is also a parameter so methods from the DrawMaze class can be
used used to find the starting position of the Runner
        runner = Runner(gameMaze, gameCanvas, runnerLocation, drawMaze)

        #Declares runner as the memory location for the Runner class
```

MAZE SURVIVOR– Computer Science NEA

```
#Initiating the class with the parameters gameMaze, gameCanvas and
runnerLocation so the Tagger's position can be updated when it moves
#drawMaze is also a parameter so methods from the DrawMaze class can be
used to find the starting position of the Tagger
tagger = Tagger(gameMaze,gameCanvas,taggerLocation,drawMaze)

#Declares gameEndChec as the memory location for the GameEndCheck class
#Initiating the class with the parameters gameCanvas and gameWindow so
the class can use Tkinter to create the timer
#self.timer is also a parameter to determine how long the timer is
#runner and tagger are parameters so methods from both class can be
used to find their positions in the maze to see if the Tagger has caught the
Runner
gameEndChec = GameEndCheck(gameWindow,
gameCanvas,self.timer,runner,tagger)
```

```
#Class which manages the operations of the runner
class Runner():
    def __init__(self, gameMaze, gameCanvas, runnerLocation, drawMaze):
        #Initiates the class

        #Creates the attribute self.runnerPosInMaze which holds the value
        stored in the gameMaze parameter
        #The gameMaze parameter will hold the contents of gameMaze from the
        GameManager class when the program is executed
```

MAZE SURVIVOR– Computer Science NEA

```
self.runnerPosInMaze = gameMaze

#Creates the attribute self.gameCanvas which holds the value stored in
the gameCanvas parameter
#The gameCanvas parameter will hold the contents of gameCanvas from the
GameManager class when the program is executed
self.gameCanvas = gameCanvas

#Creates the attribute self.runnerLocation which holds the value stored
in the runnerLocation parameter
#The runnerLocation parameter will hold the contents of runnerLocation
from the GameManager class when the program is executed
self.runnerLocation = runnerLocation

#Creates the attribute self.drawMaze which holds the value stored in
the drawMaze parameter
#The drawMaze parameter holds the contents of drawMaze from the
GameManager class when the program is executed
#This means self.drawMaze contains the DrawMaze Class so it can use
it's methods to get the runner's starting position
self.drawMaze = drawMaze

#Runs the method movement
self.movement()

#Method that gets the position of the runner in the Maze
def movement(self):

    #Using the method from the DrawMaze class, it gets the runner's
    position in the Y-axis of the maze (Row of the 2D-array)
    self.runnerPositionY = self.drawMaze.returnRunnerPositionY()

    #Using the method from the DrawMaze class, it gets the runner's
    position in the X-axis of the maze (Column of the 2D-array)
    self.runnerPositionX = self.drawMaze.returnRunnerPositionX()

    #Sets up the keybind for the W-key using the .bind function
    #This means every time the W-key is pressed it will run the method
    keyPressW
    self.gameCanvas.bind("<w>", self.keyPressW)
    self.gameCanvas.bind("<W>", self.keyPressW)

    #Sets up the keybind for the S-key using the .bind function
    #This means every time the S-key is pressed it will run the method
    keyPressS
    self.gameCanvas.bind("<s>", self.keyPressS)
    self.gameCanvas.bind("<S>", self.keyPressS)

    #Sets up the keybind for the A-key using the .bind function
    #This means every time the A-key is pressed it will run the method
    keyPressA
```

MAZE SURVIVOR– Computer Science NEA

```
self.gameCanvas.bind("<a>", self.keyPressA)
self.gameCanvas.bind("<A>", self.keyPressA)

#Sets up the keybind for the D-key using the .bind function
#This means every time the D-key is pressed it will run the method
keyPressD
self.gameCanvas.bind("<d>", self.keyPressD)
self.gameCanvas.bind("<D>", self.keyPressD)

#Activates the keybinds using the .focus_set function
self.gameCanvas.focus_set()

#Method that disables the Runner's movement keys when the game ends
def stopRunner(self):
    #Deactivates the keybind for the W-key using the .unbind function so
the runner can no longer move up
    self.gameCanvas.unbind("<w>")
    self.gameCanvas.unbind("<W>")

    #Deactivates the keybind for the S-key using the .unbind function so
the runner can no longer move down
    self.gameCanvas.unbind("<s>")
    self.gameCanvas.unbind("<S>")

    #Deactivates the keybind for the A-key using the .unbind function so
the runner can no longer move left
    self.gameCanvas.unbind("<a>")
    self.gameCanvas.unbind("<A>")

    #Deactivates the keybind for the D-key using the .unbind function so
the runner can no longer move right
    self.gameCanvas.unbind("<d>")
    self.gameCanvas.unbind("<D>")

#Method that is run when the W-key is pressed
def keyPressW(self,event):

    #Checks if the runner can move up
    #If there is a path ahead, the runner moves up the maze and the place
where it just was is set to a path
    if self.runnerPosInMaze[self.runnerPositionY-1][self.runnerPositionX]
== 'P':
        self.runnerPosInMaze[self.runnerPositionY-1][self.runnerPositionX]
= 'runner'
        self.runnerPosInMaze[self.runnerPositionY][self.runnerPositionX] =
'P'
```

MAZE SURVIVOR– Computer Science NEA

```
#Moves the Runner sprite up using the Tkinter function .move by the
coordinate (0, -15) (15 is the constant used for square size which is used in
DrawMaze.py)
    self.gameCanvas.move(self.runnerLocation, 0, -15)

#Updates the Runner position in the Y-axis of the maze (Row of the
2D-array)
    self.runnerPositionY = self.runnerPositionY - 1

#Method that is run when the S-key is pressed
def keyPressS(self,event):

    #Checks if the runner can move down
    #If there is a path ahead, the runner moves down the maze and the place
where it just was is set to a path in the 2D-array
    if self.runnerPosInMaze[self.runnerPositionY+1][self.runnerPositionX]
== 'P':
        self.runnerPosInMaze[self.runnerPositionY+1][self.runnerPositionX]
= 'runner'
        self.runnerPosInMaze[self.runnerPositionY][self.runnerPositionX] =
'P'

    #Moves the Runner sprite down using the Tkinter function .move by
the coordinate (0, 15) (15 is the constant used for square size which is used
in DrawMaze.py)
    self.gameCanvas.move(self.runnerLocation, 0, 15)

#Updates the Runner position in the Y-axis of the maze (Row of the
2D-array)
    self.runnerPositionY = self.runnerPositionY + 1

#Method that is run when the A-key is pressed
def keyPressA(self,event):

    #Checks if the runner can move left
    #If there is a path ahead, the runner moves left in the maze and the
place where it just was is set to a path in the 2D-array
    if self.runnerPosInMaze[self.runnerPositionY][self.runnerPositionX-1]
== 'P':
        self.runnerPosInMaze[self.runnerPositionY][self.runnerPositionX-1]
= 'runner'
        self.runnerPosInMaze[self.runnerPositionY][self.runnerPositionX] =
'P'

    #Moves the Runner sprite left using the Tkinter function .move by
the coordinate (-15, 0) (15 is the constant used for square size which is used
in DrawMaze.py)
    self.gameCanvas.move(self.runnerLocation, -15,0)
```

MAZE SURVIVOR– Computer Science NEA

```
#Updates the Runner position in the X-axis of the maze (Column of
the 2D-array)
    self.runnerPositionX = self.runnerPositionX - 1

#Method that is run when the D-key is pressed
def keyPressD(self,event):

    #Checks if the runner can move right
    #If there is a path, the runner moves right in the maze and the place
    where it just was is set to a path in the 2D-array
    if self.runnerPosInMaze[self.runnerPositionY][self.runnerPositionX+1]
== 'P':
        self.runnerPosInMaze[self.runnerPositionY][self.runnerPositionX+1]
= 'runner'
        self.runnerPosInMaze[self.runnerPositionY][self.runnerPositionX] =
'P'

    #Moves the Runner sprite right using the Tkinter function .move by
the coordinate (15, 0) (15 is the constant used for square size which is used
in DrawMaze.py)
    self.gameCanvas.move(self.runnerLocation, 15,0)

    #Updates the Runner position in the X-axis of the maze (Column of
the 2D-array)
    self.runnerPositionX = self.runnerPositionX + 1

#Method that returns the runner's position in the Y-axis of the maze (Row
of the 2D-array)
def returnRunnerPositionY(self):
    return self.runnerPositionY

#Method that returns the runner's position in the X-axis of the maze
(Column of the 2D-array)
def returnRunnerPositionX(self):
    return self.runnerPositionX

class Tagger():
    def __init__(self,gameMaze, gameCanvas, taggerLocation, drawMaze):
        #Initiates the class

        #Creates the attribute self.taggerPosInMaze which holds the value
stored in the gameMaze parameter
        #The gameMaze parameter will hold the contents of gameMaze from the
GameManager class when the program is executed
        self.taggerPosInMaze = gameMaze
```

MAZE SURVIVOR– Computer Science NEA

```
#Creates the attribute self.runnerPosInMaze which holds the value
stored in the runnerLocation parameter
#The gameCanvas parameter will hold the contents of gameCanvas from the
GameManager class when the program is executed
self.gameCanvas = gameCanvas

#Creates the attribute self.taggerLocation which holds the value stored
in the taggerLocation parameter
#The taggerLocation parameter will hold the contents of runnerLocation
from the GameManager class when the program is executed
self.taggerLocation = taggerLocation

#Creates the attribute self.drawMaze which holds the value stored in
the drawMaze parameter
#The drawMaze parameter holds the contents of drawMaze from the
GameManager class when the program is executed
#This means self.drawMaze contains the DrawMaze Class so it can use
it's methods to get the runner's starting position in the Maze
self.drawMaze = drawMaze

#Runs the method movement
self.movement()

def movement(self):

    #Using the method from the DrawMaze class, it gets the tagger's
    position in the Y-axis of the maze (Row of the 2D-array)
    self.taggerPositionY = self.drawMaze.returnTaggerPositionY()

    #Using the method from the DrawMaze class, it gets the tagger's
    position in the X-axis of the maze (Column of the 2D-array)
    self.taggerPositionX = self.drawMaze.returnTaggerPositionX()

    #Sets up the keybind for the I-key using the .bind function
    #This means every time the I-key is pressed it will run the method
    keyPressI
    self.gameCanvas.bind("<i>", self.keyPressI)
    self.gameCanvas.bind("<I>", self.keyPressI)

    #Sets up the keybind for the K-key using the .bind function
    #This means every time the K-key is pressed it will run the method
    keyPressK
    self.gameCanvas.bind("<k>", self.keyPressK)
    self.gameCanvas.bind("<K>", self.keyPressK)

    #Sets up the keybind for the J-key using the .bind function
    #This means every time the J-key is pressed it will run the method
    keyPressJ
    self.gameCanvas.bind("<j>", self.keyPressJ)
    self.gameCanvas.bind("<J>", self.keyPressJ)
```

MAZE SURVIVOR– Computer Science NEA

```
#Sets up the keybind for the L-key using the .bind function
#This means every time the L-key is pressed it will run the method
keyPressL
    self.gameCanvas.bind("<l>", self.keyPressL)
    self.gameCanvas.bind("<L>", self.keyPressL)

#Activates the keybinds using the .focus_set function
self.gameCanvas.focus_set()

def stopTagger(self):
    #Deactivates the keybind for the I-key using the .unbind function so
the tagger can no longer move up
    self.gameCanvas.unbind("<i>")
    self.gameCanvas.unbind("<I>")

    #Deactivates the keybind for the K-key using the .unbind function so
the tagger can no longer move down
    self.gameCanvas.unbind("<k>")
    self.gameCanvas.unbind("<K>")

    #Deactivates the keybind for the J-key using the .unbind function so
the runner can no longer move left
    self.gameCanvas.unbind("<j>")
    self.gameCanvas.unbind("<J>")

    #Deactivates the keybind for the L-key using the .unbind function so
the runner can no longer move right
    self.gameCanvas.unbind("<l>")
    self.gameCanvas.unbind("<L>")

    #Runs the method in the DrawMaze class which destroys the game window
when the game is over
    self.drawMaze.destroyGameWindow()

#Method that is run when the I-key is pressed
def keyPressI(self,event):

    #Checks if the tagger can move up
    #If there is a path or the runner ahead, the tagger moves up the maze
and the place where it just was is set to a path
    if self.taggerPosInMaze[self.taggerPositionY-1][self.taggerPositionX]
== 'P' or self.taggerPosInMaze[self.taggerPositionY-1][self.taggerPositionX] ==
'runner':
        self.taggerPosInMaze[self.taggerPositionY-1][self.taggerPositionX]
= 'tagger'
```

MAZE SURVIVOR– Computer Science NEA

```
self.taggerPosInMaze[self.taggerPositionY][self.taggerPositionX] =
'P'

#Moves the Tagger sprite up using the Tkinter function .move by the
coordinate (0, -15) (15 is the constant used for square size which is used in
DrawMaze.py)
self.gameCanvas.move(self.taggerLocation, 0, -15)

#Updates the Runner position in the Y-axis of the maze (Row of the
2D-array)
self.taggerPositionY = self.taggerPositionY - 1

#Method that is run when the K-key is pressed
def keyPressK(self,event):

    #Checks if the tagger can move down
    #If there is a path or the runner ahead, the tagger moves down the maze
and the place where it just was is set to a path
    if self.taggerPosInMaze[self.taggerPositionY+1][self.taggerPositionX]
== 'P' or self.taggerPosInMaze[self.taggerPositionY+1][self.taggerPositionX] ==
'runner':
        self.taggerPosInMaze[self.taggerPositionY+1][self.taggerPositionX]
= 'tagger'
        self.taggerPosInMaze[self.taggerPositionY][self.taggerPositionX] =
'P'

    #Moves the Tagger sprite up using the Tkinter function .move by the
coordinate (0, 15) (15 is the constant used for square size which is used in
DrawMaze.py)
    self.gameCanvas.move(self.taggerLocation, 0, 15)

    #Updates the Runner position in the Y-axis of the maze (Row of the
2D-array)
    self.taggerPositionY = self.taggerPositionY + 1

#Method that is run when the J-key is pressed
def keyPressJ(self,event):

    #Checks if the tagger can move left
    #If there is a path or the runner ahead, the tagger moves left in the
maze and the place where it just was is set to a path
    if self.taggerPosInMaze[self.taggerPositionY][self.taggerPositionX-1]
== 'P' or self.taggerPosInMaze[self.taggerPositionY][self.taggerPositionX-1] ==
'runner':
        self.taggerPosInMaze[self.taggerPositionY][self.taggerPositionX-1]
= 'tagger'
        self.taggerPosInMaze[self.taggerPositionY][self.taggerPositionX] =
'P'
```

MAZE SURVIVOR– Computer Science NEA

```
#Moves the Tagger sprite up using the Tkinter function .move by the
coordinate (-15,0) (15 is the constant used for square size which is used in
DrawMaze.py)
    self.gameCanvas.move(self.taggerLocation, -15,0)

#Updates the Tagger position in the X-axis of the maze (Column of
the 2D-array)
    self.taggerPositionX = self.taggerPositionX - 1

#Method that is run when the L-key is pressed
def keyPressL(self,event):

    #Moves the Tagger sprite up using the Tkinter function .move by the
coordinate (0, 15) (15 is the constant used for square size which is used in
DrawMaze.py)
        if self.taggerPosInMaze[self.taggerPositionY][self.taggerPositionX+1]
== 'P'or self.taggerPosInMaze[self.taggerPositionY][self.taggerPositionX+1] ==
'runner':
            self.taggerPosInMaze[self.taggerPositionY][self.taggerPositionX+1]
= 'tagger'
            self.taggerPosInMaze[self.taggerPositionY][self.taggerPositionX] =
'P'

    #Moves the Tagger sprite up using the Tkinter function .move by the
coordinate (15,0) (15 is the constant used for square size which is used in
DrawMaze.py)
        self.gameCanvas.move(self.taggerLocation, 15,0)

    #Updates the Tagger position in the X-axis of the maze (Column of
the 2D-array)
        self.taggerPositionX = self.taggerPositionX + 1

    #Method that returns the tagger's position in the Y-axis of the maze (Row
of the 2D-array)
    def returnTaggerPositionY(self):
        return self.taggerPositionY

    #Method that returns the tagger's position in the X-axis of the maze
(Column of the 2D-array)
    def returnTaggerPositionX(self):
        return self.taggerPositionX

class GameEndCheck():
    def __init__(self, gameWindow, gameCanvas, timer, runner, tagger):
        #Initiates the class

        #Creates the attribute self.gameWindow which holds the value stored in
the gameWindow parameter
```

MAZE SURVIVOR- Computer Science NEA

```
#The gameWindow parameter will hold the contents of gameWindow from the
GameManager class when the program is executed
self.gameWindow = gameWindow

#Creates the attribute self.timer which holds the value stored in the
timer parameter
#The timer parameter will hold the contents of timer from the
GameManager class when the program is executed
self.timer = timer

#Creates the attribute self.gameCanvas which holds the value stored in
the gameCanvas parameter
#The gameCanvas parameter will hold the contents of gameCanvas from the
GameManager class when the program is executed
self.gameCanvas = gameCanvas

#Runs the method createTimer
self.createTimer()

#Creates the attribute self.runner which holds the value stored in the
runner parameter
#The runner parameter holds the contents of runner from the GameManager
class when the program is executed
#This means self.runner contains the Runner Class so it can use it's
methods to get the runner's current position in the maze
self.runner = runner

#Creates the attribute self.tagger which holds the value stored in the
tagger parameter
#The tagger parameter holds the contents of tagger from the GameManager
class when the program is executed
#This means self.tagger contains the Tagger Class so it can use it's
methods to get the taggers's current position in the maze
self.tagger = tagger

#Creates the attribute self.gameEnd which is set to False and will
become True if the conditions are met for the game to end
self.gameEnd = False

#Runs the method runTimer, that uses self.timer as the parameter for
timeleft in the runTimer method
self.runTimer(self.timer)

#Method the creates the GUI for the timer using Tkinter
def createTimer(self):
    #Using the Label and .place functions, it creates the Timer button is
    made which will display the value of self.timer to show how many seconds are
    left
    self.timerButton = tk.Label(self.gameWindow, text = str(self.timer), bg
    = "black", height = 3, width = 10, font = "Arial", fg = "white")
    self.timerButton.place(x = 1100, y = 30)
```

MAZE SURVIVOR- Computer Science NEA

```
#Method that uses recursion to operate the timer
#The parameter timeleft will hold the value stored in self.timer
def runTimer(self,timeleft):
    #Checks to see if the timer has hit 0 or if the value gameEnd is True
    if timeleft > 0 and self.gameEnd == False:
        #Takes away 1 from time left
        timeleft = timeleft - 1

        #Runs the method winCheck
        self.winCheck()

        #Updates the timer to show the new value
        self.timerButton.config(text = str(timeleft))

        #Using the Tkinter function.after, there will be a 1 second pause
        (1000 milliseconds) and then will run the method runTimer with the parameter of
        timeleft

        self.gameCanvas.after(1000, self.runTimer, timeleft)

    #Else the tagger has caught the player or the timer hits zero
    else:
        #Sets the value of self.gameEnd to True
        self.gameEnd = True

        #Removes sets the timer to show nothing and then the timer is
        removed from the screen using the .destroy function
        self.timerButton.config(text = "")
        self.timerButton.destroy()

        #Runs the method in the Runner class stopRunner so the Runner can
        no longer move
        self.runner.stopRunner()

        #Runs the method in the Tagger class stopTagger so the Tagger can
        no longer move
        self.tagger.stopTagger()

        #Runs the method runnerWins which will display the Runner Wins Menu
        self.runnerWins()

#Method that is run every second to check if the tagger has found the
player
def winCheck(self):

    #Using the method returnRunnerPositionY from the Runner Class, which
    will store the runner's position in the Y-axis of the maze into runnerPositionY
    runnerPositionY = self.runner.returnRunnerPositionY()
```

MAZE SURVIVOR- Computer Science NEA

```
#Using the method returnRunnerPositionX from the Runner Class, which
will store the runner's position in the X-axis of the maze into runnerPositionX
runnerPositionX = self.runner.returnRunnerPositionX()

#Using the method returnTaggerPositionY from the Tagger Class, which
will store the tagger's position in the Y-axis of the maze into taggerPositionY
taggerPositionY = self.tagger.returnTaggerPositionY()

#Using the method returnTaggerPositionX from the Tagger Class, which
will store the tagger's position in the X-axis of the maze into taggerPositionX
taggerPositionX = self.tagger.returnTaggerPositionX()

#Checks to see if the tagger's position in the maze is equal to the
position to the runner.
if taggerPositionY == runnerPositionY and taggerPositionX ==
runnerPositionX:
    #If equal then the game is over
    #Sets the value of self.gameEnd to True
    self.gameEnd = True

    #Runs the method in the Runner class stopRunner so the Runner can
no longer move
    self.runner.stopRunner()

    #Runs the method in the Tagger class stopTagger so the Tagger can
no longer move
    self.tagger.stopTagger()

    #Runs the method taggerWins which will display the Tagger Wins Menu
    self.taggerWins()

def runnerWins(self):
    #Creating the Runner Wins Menu window using Tkinter, while also setting
the window name and it's dimensions
    self.winMenu = tk.Tk()
    self.winMenu.title('Maze Survivor')
    self.winMenu.geometry("1280x800")

    #Setting up the background of Runner wins menu
    #Using the PhotoImage function RunnerWins.png is stored in
RunnerWinImage
    RunnerWinImage = PhotoImage(file = "C:\\\\PROJECT\\\\GUI\\\\RunnerWins.png")

    #Using the Label and .place functions, the RunnerWins label is created
which is the image stored in RunnerWinImage displayed in the window of the
Runner Wins Menu
    RunnerWins = Label(self.winMenu, image = RunnerWinImage)
    RunnerWins.place(x = 0, y = 0)

    #Setting up the Menu button
    #Using the PhotoImage function MENU.png is stored in MenuGUI
```

MAZE SURVIVOR- Computer Science NEA

```
MenuGUI = PhotoImage(file = "C:\\PROJECT\\GUI\\MENU.png")

#Using the Button and .place functions the Menu button is created which
is displayed in the window of the Runner Wins Menu
#The image of the button is the image stored in MenuGUI
#When the button is pressed it runs the method backToMenu
MenuButton = Button(self.winMenu, image = MenuGUI, command =
self.backToMenu)
MenuButton.place(x = 50, y = 300)

#Setting up the Quit button
#Using the PhotoImage function QuitButton.png is stored in QuitGUI
QuitGUI = PhotoImage(file = "C:\\PROJECT\\GUI\\QuitButton.png")

#Using the Button and .place functions the Menu button is created which
is displayed in the window of the Runner Wins Menu
#The image of the button is the image stored in QuitGUI
#When the button is pressed it runs the method Quit
QuitButton = Button(self.winMenu, image = QuitGUI, command = self.Quit)
QuitButton.place(x = 730 , y = 300)

#Tkinter function that runs the while loop to keep the Runner Wins Menu
window responsive
self.winMenu.mainloop()

def taggerWins(self):
    #Creating the Tagger Wins Menu window using Tkinter, while also setting
the window name and it's dimensions
    self.winMenu = tk.Tk()
    self.winMenu.title('Maze Survivor')
    self.winMenu.geometry("1280x800")

    #Setting up the background of Tagger Wins Menu
    #Using the PhotoImage function TaggerWins.png is stored in
TaggerWinImage
    TaggerWinImage = PhotoImage(file = "C:\\PROJECT\\GUI\\TaggerWins.png")

    #Using the Label and .place functions, the TaggerWins label is created
which is the image stored in TaggerWinImage displayed in the window of the
Tagger Wins Menu
    TaggerWins = Label(self.winMenu, image = TaggerWinImage)
    TaggerWins.place(x = 0, y = 0)

    #Setting up the Menu button
    #Using the PhotoImage function MENU.png is stored in MenuGUI
    MenuGUI = PhotoImage(file = "C:\\PROJECT\\GUI\\MENU.png")
```

MAZE SURVIVOR– Computer Science NEA

```
#Using the Button and .place functions the Menu button is created which
is displayed in the window of the Tagger Wins Menu
#The image of the button is the image stored in MenuGUI
#When the button is pressed it runs the method backToMenu
MenuButton = Button(self.winMenu, image = MenuGUI, command =
self.backToMenu)
MenuButton.place(x = 50, y = 300)

#Using the Button and .place functions the Menu button is created which
is displayed in the window of the Tagger Wins Menu
#The image of the button is the image stored in QuitGUI
#When the button is pressed it runs the method Quit
QuitGUI = PhotoImage(file = "C:\\PROJECT\\GUI\\QuitButton.png")
QuitButton = Button(self.winMenu, image = QuitGUI, command = self.Quit)
QuitButton.place(x = 730 , y = 300)

#Tkinter function that runs the while loop to keep the Runner Wins Menu
window responsive
self.winMenu.mainloop()

def backToMenu(self):
    self.winMenu.destroy()
    backToMenu = startMenu()

#Method that destroys the Runner Wins Menu if the Quit button is pressed
using the .destroy function
def Quit(self):
    self.winMenu.destroy()

#Declares menu as the memory location for startMenu Class, which initiates the
class and creates the main menu
menu = startMenu()
```

MAZE SURVIVOR– Computer Science NEA

Testing

End Product Testing:

Please refer to the **16115_7517-C_6122_MazeSurvivorTesting.mp4**

Test	Test Criteria	Expected Outcome	Outcome (Time stap for video)
Main Menu Opens	Executing the Main.py file to start the program	A window is created which has the title “Maze Survivor” which displays the Main Menu	0:00 – 0:09
Close the Main Menu	Pressing the Quit button in the Main Menu	The Main menu will close	0:10 – 0:15
Set the game difficulty to Easy	Pressing the Easy Button in the Main Menu	Sets the value of the timer to 60	0:16 – 0:18
Set the game difficulty to Medium	Pressing the Medium Button in the Main Menu	Sets the value of the timer to 120	0:19 – 0:22
Set the game difficulty to Hard	Pressing the Hard Button in the Main Menu	Sets the value of the timer to 180	0:24 – 0:27
Start Game	Pressing the Play Game button in the Main Menu. This also makes sure that MazeGeneration.py and DrawMaze.py are working and experience no issues	Main menu closes and a new window opens. The new window will display the Maze, Timer and the Runner & Tagger sprites.	0:29 – 0:35
Generate a text file which holds the maze	Having file explorer open when starting the game	A text file called “GeneratedMaze.txt” should appear	0:36 – 0:47
Runner moves up a path	Pressing the W key when a path is ahead	The Runner sprite will move up one cell in the maze	0:48 – 0:50
Runner tries to move up into a wall	Pressing the W key when a wall is ahead	The Runner sprite will not move up one cell in the maze	0:51 – 0:55
Runner moves down a path	Pressing the S key when a path is ahead	The Runner sprite will move down one cell in the maze	0:57 – 1:00
Runner tries to move	Pressing the S key when a wall is ahead	The Runner sprite will not move down one cell in the maze	1:01 – 1:04

MAZE SURVIVOR– Computer Science NEA

down into a wall			
Runner moves left when a path is ahead	Pressing the A key when a path is ahead	The Runner sprite will move left one cell in the maze	1:06 - 1:08
Runner tries to move left into a wall	Pressing the A key when a wall is ahead	The Runner sprite will not move left one cell in the maze	1:09 - 1:12
Runner moves right when a path is ahead	Pressing the D key when a path is ahead	The Runner sprite will move right one cell in the maze	1:16 - 1:18
Runner tries to move right into a wall	Pressing the D key when a wall is ahead	The Runner sprite will not move right one cell in the maze	1:19 - 1:20
Tagger moves up a path	Pressing the I key when a path is ahead	The Tagger sprite will move up one cell in the maze	1:25 - 1:27
Tagger tries to move up into a wall	Pressing the I key when a wall is ahead	The Tagger sprite will not move up one cell in the maze	1:28 - 1:30
Tagger moves down a path	Pressing the K key when a path is ahead	The Tagger sprite will move down one cell in the maze	1:31 - 1:33
Tagger tries to move down into a wall	Pressing the K key when a wall is ahead	The Tagger sprite will not move down one cell in the maze	1:34 - 1:36
Tagger moves right when a path is ahead	Pressing the L key when a path is ahead	The Tagger sprite will move right one cell in the maze	1:37 - 1:39
Tagger tries moves right when a wall is ahead	Pressing the L key when a wall is ahead	The Tagger sprite will not move right one cell in the maze	1:39 - 1:42
Tagger moves right when a path is ahead	Pressing the J key when a path is ahead	The Tagger sprite will move left one cell in the maze	1:42 - 1:44
Tagger tries moves right when a wall is ahead	Pressing the J key when a wall is ahead	The Tagger sprite will not move left one cell in the maze	1:45 - 1:46

MAZE SURVIVOR– Computer Science NEA

Timer counts down by 1 second	Run the game and see if the timer is working properly	If the Runner has not been caught the timer should work as expected	1:50 – 1:55
Runner Wins	Play the game normally until the timer reaches zero	If the timer reaches zero and the Runner has not been caught, the game window will close, and a new menu will open which displays the message “Runner Wins!” with two buttons to go back to the main menu or to quit	1:56 – 2:06
Tagger Wins	Play the game normally until the Tagger catches the Runner	If the Tagger sprite is in the exact same cell as the Runner sprite, the game window will close, and a new menu will open which displays the message “Tagger Wins!” with two buttons to go back to the main menu or to quit	2:07 – 2:14
Go back to the Main Menu from the Runner Wins Menu	Press the Menu button in the Runner Wins Menu	The Runner Wins Menu will close, and the Main Menu will open	2:15 – 2:17
Go back to the Main Menu from the Tagger Wins Menu	Press the Menu button in the Tagger Wins Menu	The Tagger Wins Menu will close, and the Main Menu will open	2:18 – 2:21
Close the Runner Wins Menu	Press the Quit button in the Runner Wins Menu	The Runner Wins Menu will close	2:26 – 2:32
Close the Tagger Wins Menu	Press the Quit button in the tagger	The Tagger Wins Menu will close	2:33 – 2:38

MAZE SURVIVOR– Computer Science NEA

Evaluation

Application Evaluation:

Objective	Did I successful meet the objective?	How was this achieved?
A button to start the game in the Main Menu	Yes	By using Tkinter, I was able to make a button which started the game when pressed
A button that closes the Main Menu	Yes	By using Tkinter, I was able to make a button which closed the main menu when pressed
A button that sets that sets the game difficulty to Easy	Yes	By using Tkinter, I was able to make a button which set the game difficulty to easy, in turn setting the time the Runner must survive to 60 seconds
A button that sets that sets the game difficulty to Medium	Yes	By using Tkinter, I was able to make a button which set the game difficulty to medium, in turn setting the time the Runner must survive to 120 seconds
A button that sets that sets the game difficulty to Hard	Yes	By using Tkinter, I was able to make a button which set the game difficulty to Hard, in turn setting the time the Runner must survive to 180 seconds
Generate a maze using a modified version of Prim's algorithm	Yes	I was able to do this by setting all the 2 outer cells as "barriers" around the outside of the maze. That way during the maze generation, no frontier cells we're selected from outside the confinement of the barrier
Take the maze from a 2d-array format and write it into a text document	Yes	By using file handling, I was able to take the maze from a 2d array format into a text document
Tkinter then reads the text document and then draws the maze	Yes	I struggled to find a way for Tkinter to read a text document and then draw the maze. My work around was to have it so in the DrawMaze class the text file be converted back into a 2d array and then from there, use Tkinter to draw the maze as it went through each cell in the Maze
Spawn in the runner and the tagger at random starting points in the maze	Yes	By using Python's Random library, A random Y and X coordinate are picked. If that cell is a path, then the Runner is put in that location, and the same processes is repeated for the Tagger, making sure the random starting cell selected is also not occupied by the Runner

MAZE SURVIVOR– Computer Science NEA

A timer that counts from time set by the game difficulty to zero	Yes	By using Tkinter, I was able to produce a label which displayed how much timer the Runner had left to survive. Each second the label would update with a second deducted from the timer. This process was done using a recursive method
Each second a method is run to see if the tagger position in the maze is the same as the Runner position	Yes	I was able to do this by having using a recursive method which each second would check if the Tagger's Y and X coordinate equalled the Runner's
If the tagger gets the Runner, it displays a message saying, "Tagger Wins!"	Yes	By using Tkinter, I was able to make a menu which would open when the Tagger's Y and X coordinate equalled the Runner's.
If the Runner survives display a message saying, "Runner Wins!"	Yes	By using Tkinter, I was able to make a menu which would open when the timer equalled zero
A button that takes you back to the Main Menu	Yes	By using Tkinter, I was able to make a button which took you back to the main menu when pressed in both the Runner Wins and Tagger Wins Menu
A button that quits the game	Yes	By using Tkinter, I was able to make a button which when pressed closed both the Runner Wins and Tagger Wins Menu
Runner moves up when the W key is pressed and does not move through walls	Yes	By using the Tkinter bind function, I was able to run a method that moved the Runner sprite up on cell when the W key was pressed. An IF statement in the method checked to see if the cell ahead a wall was if the. If it was the Runner sprite didn't move
Runner moves down when the S key is pressed and does not move through walls	Yes	By using the Tkinter bind function, I was able to run a method that moved the Runner sprite down on cell when the S key was pressed. An IF statement in the method checked to see if the cell ahead a wall. If it was, the Runner sprite didn't move
Runner moves left when the A key is pressed and does not move through walls	Yes	By using the Tkinter bind function, I was able to run a method that moved the Runner sprite left on cell when the A key was pressed. An IF statement in the method checked to see if the cell ahead a wall. If it was, the Runner sprite didn't move.
Runner moves right when the D key is	Yes	By using the Tkinter bind function, I was able to run a method that moved the Runner sprite right on cell when the D key was pressed. An IF

MAZE SURVIVOR– Computer Science NEA

pressed and does not move through walls		statement in the method checked to see if the cell ahead a wall. If it was, the Runner sprite didn't move.
Tagger moves up when the I key is pressed and does not move through walls	Yes	By using the Tkinter bind function, I was able to run a method that moved the Tagger sprite right on cell when the I key was pressed. An IF statement in the method checked to see if the cell ahead a wall. If it was, the Tagger sprite didn't move.
Tagger moves down when the K key is pressed and does not move through walls	Yes	By using the Tkinter bind function, I was able to run a method that moved the Tagger sprite right on cell when the K key was pressed. An IF statement in the method checked to see if the cell ahead a wall. If it was, the Tagger sprite didn't move.
Tagger moves left when the J key is pressed and does not move through walls	Yes	By using the Tkinter bind function, I was able to run a method that moved the Tagger sprite right on cell when the J key was pressed. An IF statement in the method checked to see if the cell ahead a wall. If it was, the Tagger sprite didn't move.
Tagger moves right when the L key is pressed and does not move through walls	Yes	By using the Tkinter bind function, I was able to run a method that moved the Tagger sprite right on cell when the L key was pressed. An IF statement in the method checked to see if the cell ahead a wall. If it was, the Tagger sprite didn't move.

End User Feedback:

Name of End User:	What did you think of Maze Survivor?
Zach	"The game is fun to play; however, it is very easy to find yourself in a dead end whilst the other player is nearby"

MAZE SURVIVOR– Computer Science NEA

Improvements For The Future:

In the future, I can make improvements to the Maze generation, improving further so there are no dead ends in the maze, this way it makes the game fairer for the Runner who may find themselves at a point in the Maze where they cannot escape the Tagger. This could make the game more exciting as the Runner frantically tries to outsmart the Tagger. To achieve this, I would need to program my own version of Prim's algorithm which would prune dead ends.

Another improvement that I could make is adding an AI mode to Maze Survivor meaning two players are not required to enjoy the game. To achieve this, I would need to investigate path finding algorithms such as Dijkstra's or A*^s.

Finally, Maze Survivor could also have be multiplayer application, allowing the game to played on two separate computers, with one being the Runner and the other Tagger. To achieve this, I would need a server that allow two clients to communicate with each other so the application can be run across the internet. This would require User Datagram Protocol (UDP) which would be used by the clients that would communicate actions such as moving forward one cell in the maze and the server would receive the communication and carry out the transmission so that the other client can see that the other player has moved in the maze. Having the game be a multiplayer application also allows for a leaderboard. This leaderboard can be managed by using a database which gets updates every time a game is played. Players could then search up other players. This data would be shown the player by abstracting SQL statements from the player by

Overall, I am happy with how Maze Survivor turned out and have improved my programming skills and abilities as result of developing this project.

MAZE SURVIVOR– Computer Science NEA

Bibliography:

The Buckblog (created by James Buck): <https://weblog.jamisbuck.org/>

Used during my research for Maze generation algorithms.

Wikipedia: <https://www.wikipedia.org/>

Used during my research for Prim's algorithm and when looking for inspiration for my project.

GeeksforGeeks: <https://www.geeksforgeeks.org/>

Used during my research of which programming languages to use, python's libraries and learning Tkinter.

Visual Paradigm Online: <https://online.visual-paradigm.com/>

Used for designing of my flowchart, data flow and class diagrams.

Medium: <https://medium.com/>

Used to find the Pseudocode for Eller's algorithm